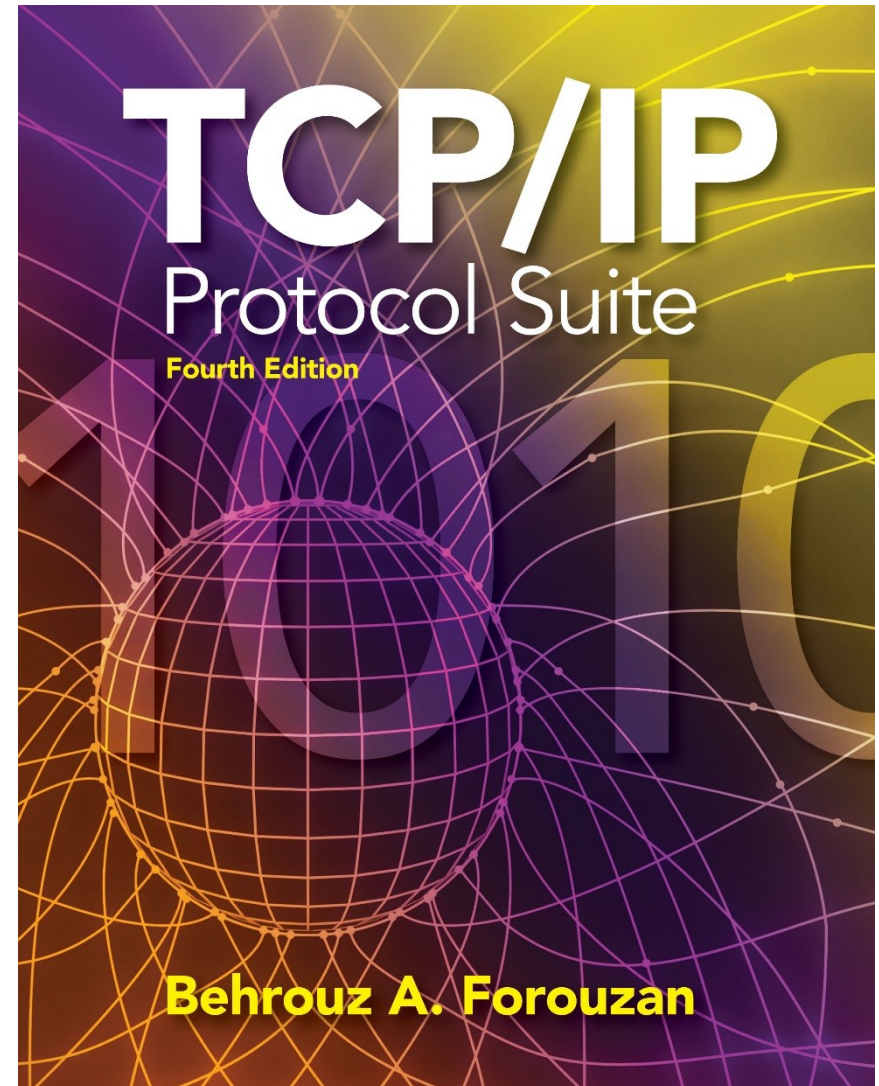


Chapter 15

Transmission Control Protocol (TCP)



OBJECTIVES:

- ☐ To introduce TCP as a protocol that provides reliable stream delivery service.
- ☐ To define TCP features and compare them with UDP features.
- ☐ To define the format of a TCP segment and its fields.
- ☐ To show how TCP provides a connection-oriented service, and show the segments exchanged during connection establishment and connection termination phases.
- ☐ To discuss the state transition diagram for TCP and discuss some scenarios.
- ☐ To introduce windows in TCP that are used for flow and error control.

OBJECTIVES (*continued*):

- ❑ To discuss how TCP implements flow control in which the receive window controls the size of the send window.
- ❑ To discuss error control and FSMs used by TCP during the data transmission phase.
- ❑ To discuss how TCP controls the congestion in the network using different strategies.

Chapter Outline

15.1
15.2
15.3
15.4
15.5
15.6
15.7
15.8
15.9

TCP Services

TCP Features

Segment

A TCP Connection

State Transition Diagram

Windows in TCP

Flow Control

Error Control

Congestion Control

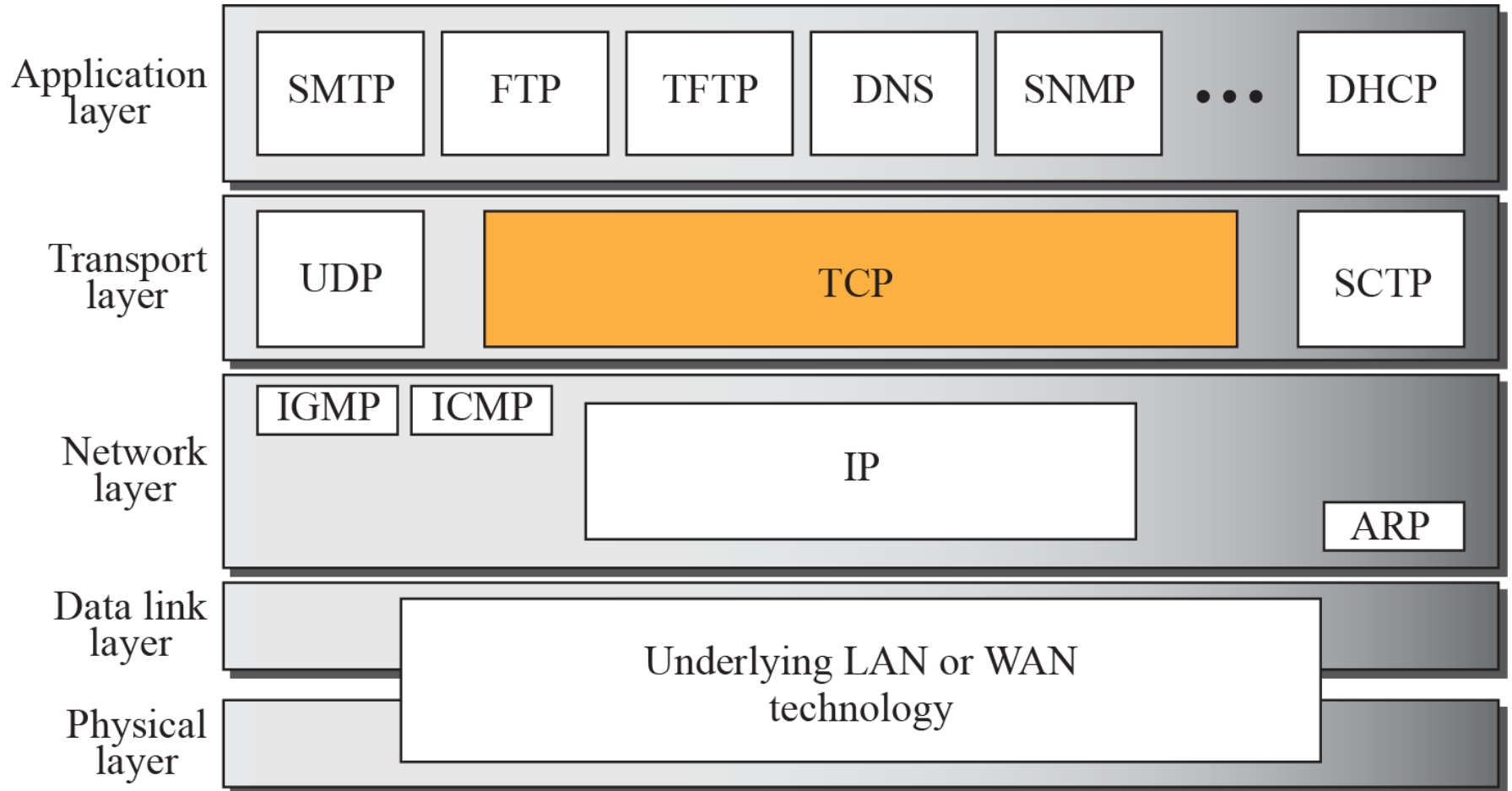
15-1 TCP SERVICES

Figure 15.1 shows the relationship of TCP to the other protocols in the TCP/IP protocol suite. TCP lies between the application layer and the network layer, and serves as the intermediary between the application programs and the network operations.

Topics Discussed in the Section

- ✓ **Process-to-Process Communication**
- ✓ **Stream Delivery Service**
- ✓ **Full-Duplex Communication**
- ✓ **Multiplexing and Demultiplexing**
- ✓ **Connection-Oriented Service**
- ✓ **Reliable Service**

Figure 15.1 *TCP/IP protocol suite*



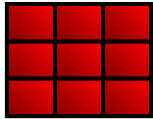


Table 15.1 *Well-known Ports used by TCP*

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20 and 21	FTP	File Transfer Protocol (Data and Control)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol

Figure 15.2 *Stream delivery*

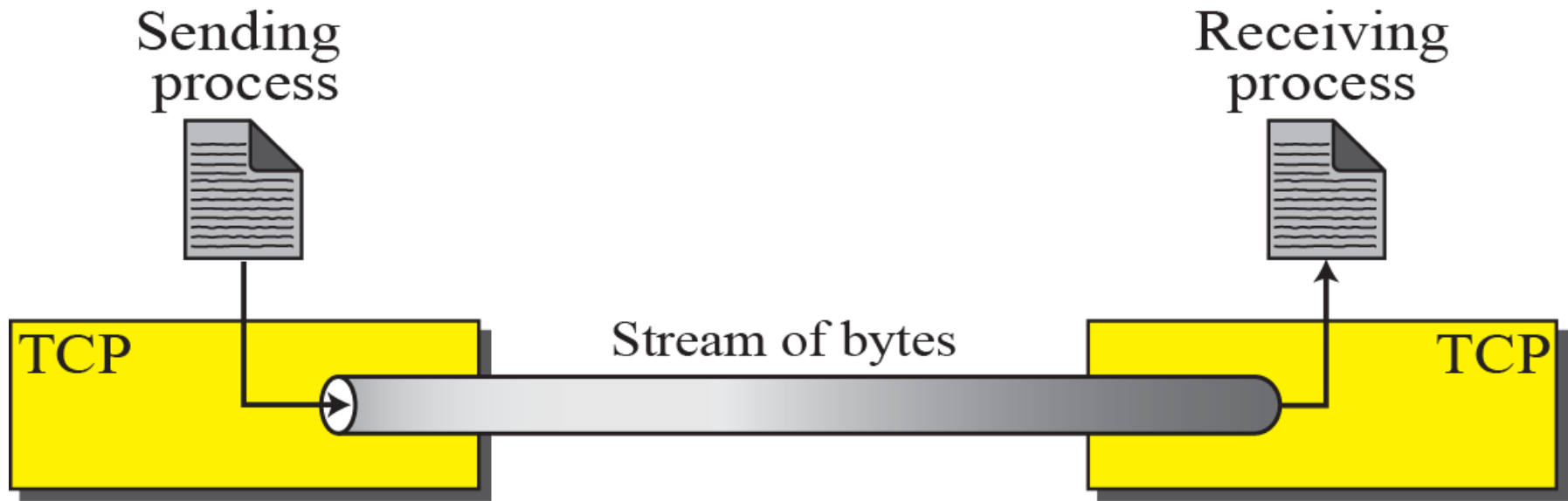


Figure 15.3 *Sending and receiving buffers*

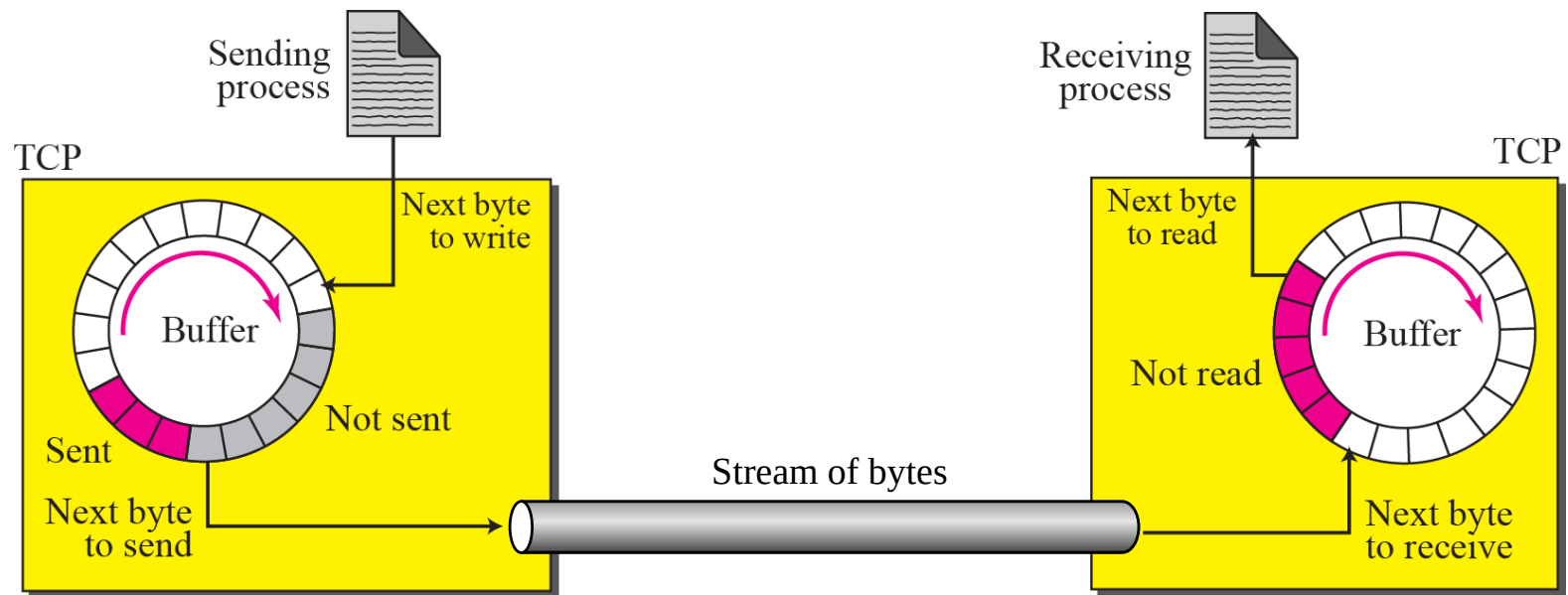
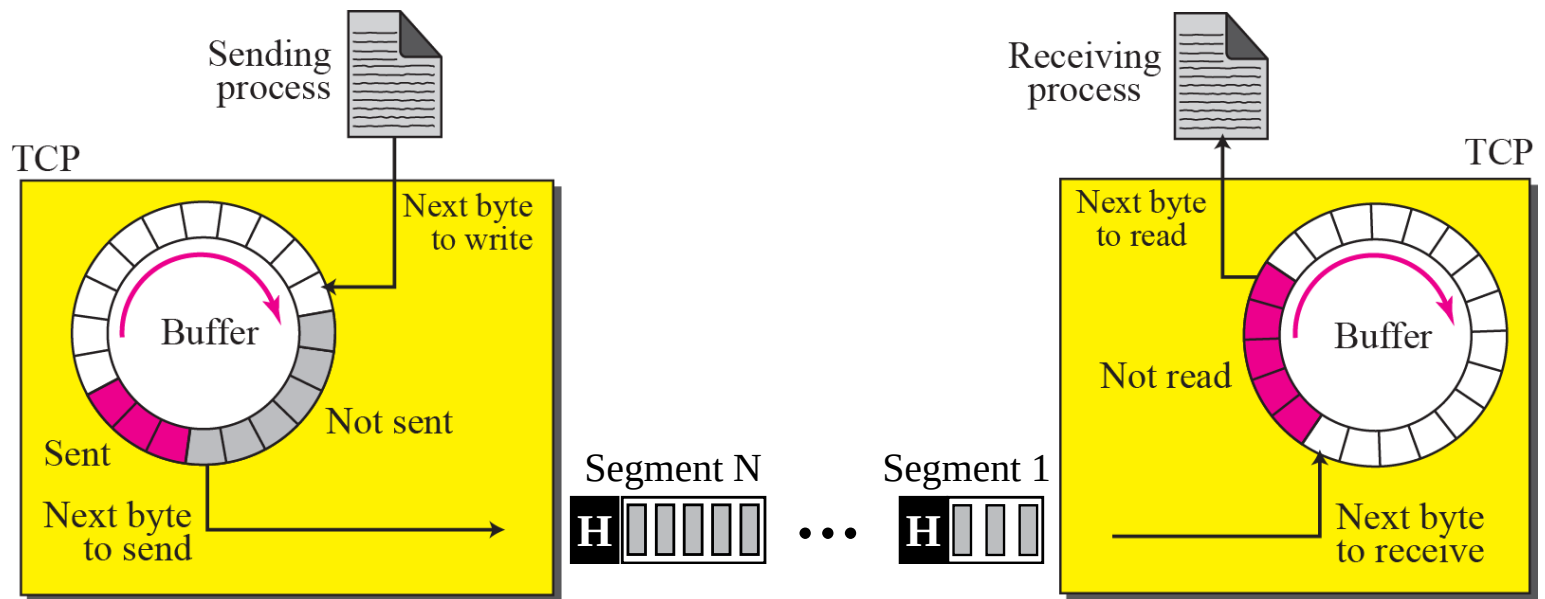


Figure 15.4 *TCP segments*



15-2 TCP FEATURES

To provide the services mentioned in the previous section, TCP has several features that are briefly summarized in this section and discussed later in detail.

Topics Discussed in the Section

- ✓ **Numbering System**
- ✓ **Flow Control**
- ✓ **Error Control**
- ✓ **Congestion Control**

Note

The bytes of data being transferred in each connection are numbered by TCP.

The numbering starts with an arbitrarily generated number.

Example 15.1

Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?

Solution

The following shows the sequence number for each segment:

Segment 1	→	Sequence Number:	10,001	Range:	10,001	to	11,000
Segment 2	→	Sequence Number:	11,001	Range:	11,001	to	12,000
Segment 3	→	Sequence Number:	12,001	Range:	12,001	to	13,000
Segment 4	→	Sequence Number:	13,001	Range:	13,001	to	14,000
Segment 5	→	Sequence Number:	14,001	Range:	14,001	to	15,000

Note

The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.

Note

The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive.

The acknowledgment number is cumulative.

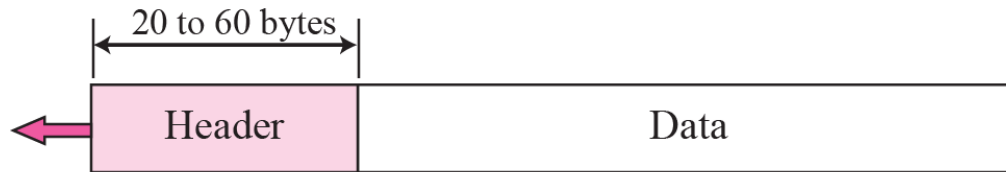
15-3 SEGMENT

Before discussing TCP in more detail, let us discuss the TCP packets themselves. A packet in TCP is called a segment.

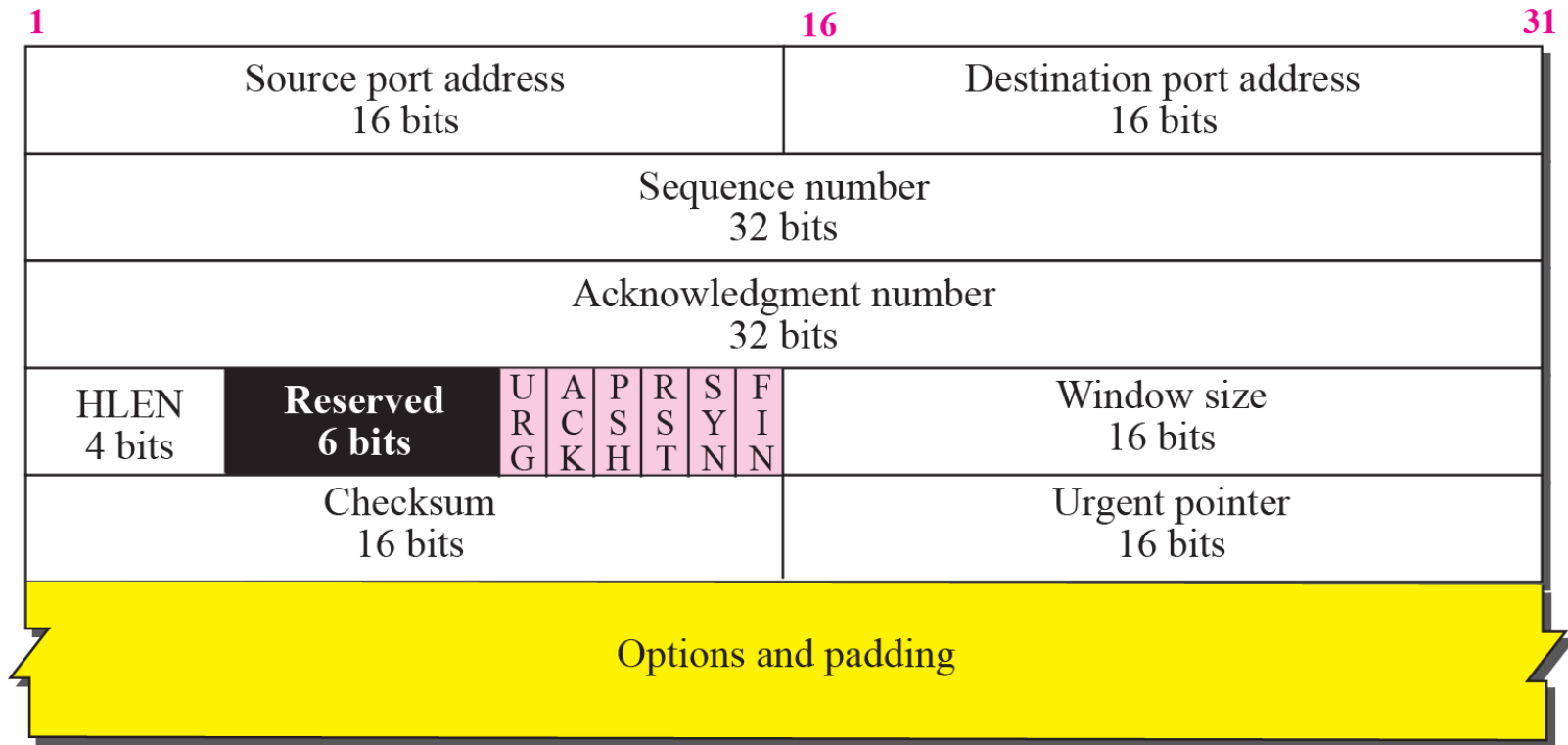
Topics Discussed in the Section

- ✓ **Format**
- ✓ **Encapsulation**

Figure 15.5 *TCP segment format*



a. Segment



b. Header

Figure 15.6 *Control field*

URG: Urgent pointer is valid

ACK: Acknowledgment is valid

PSH: Request for push

RST: Reset the connection

SYN: Synchronize sequence numbers

FIN: Terminate the connection

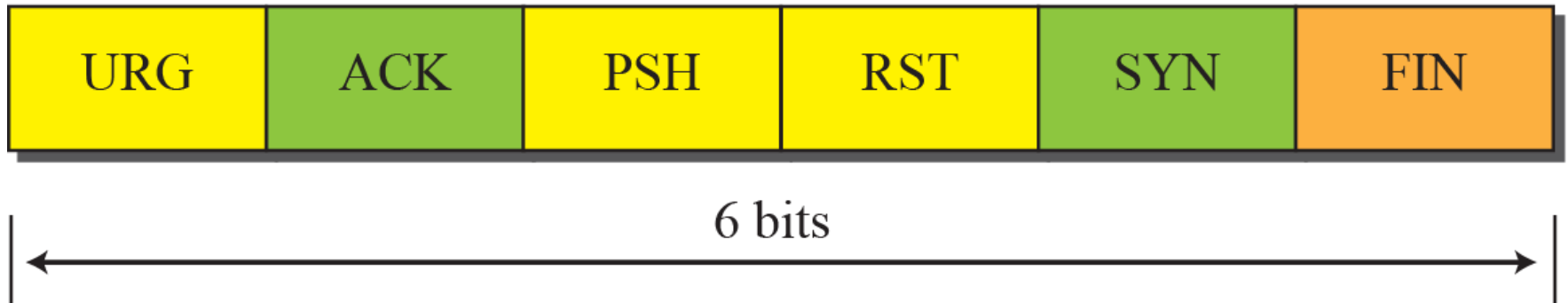
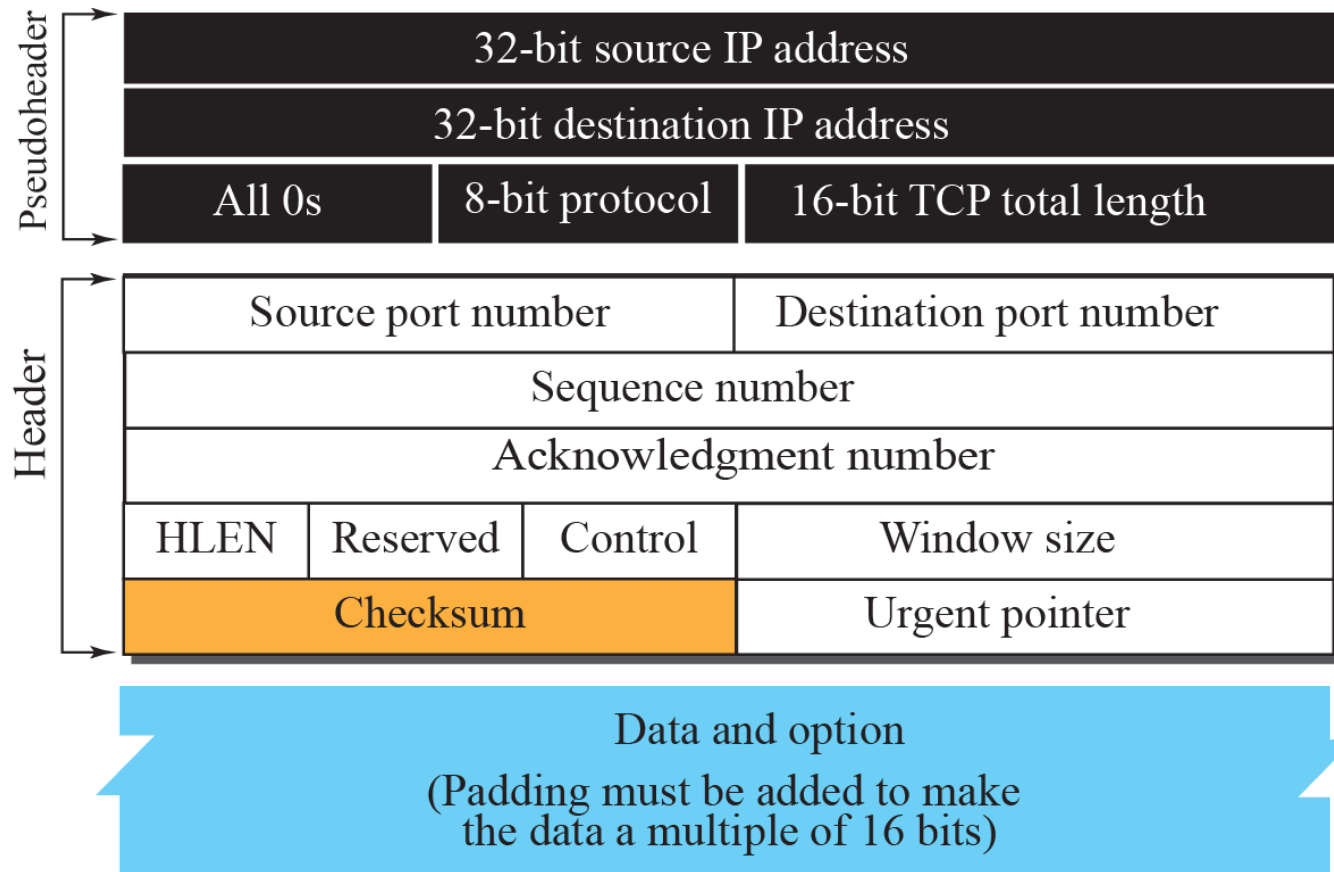


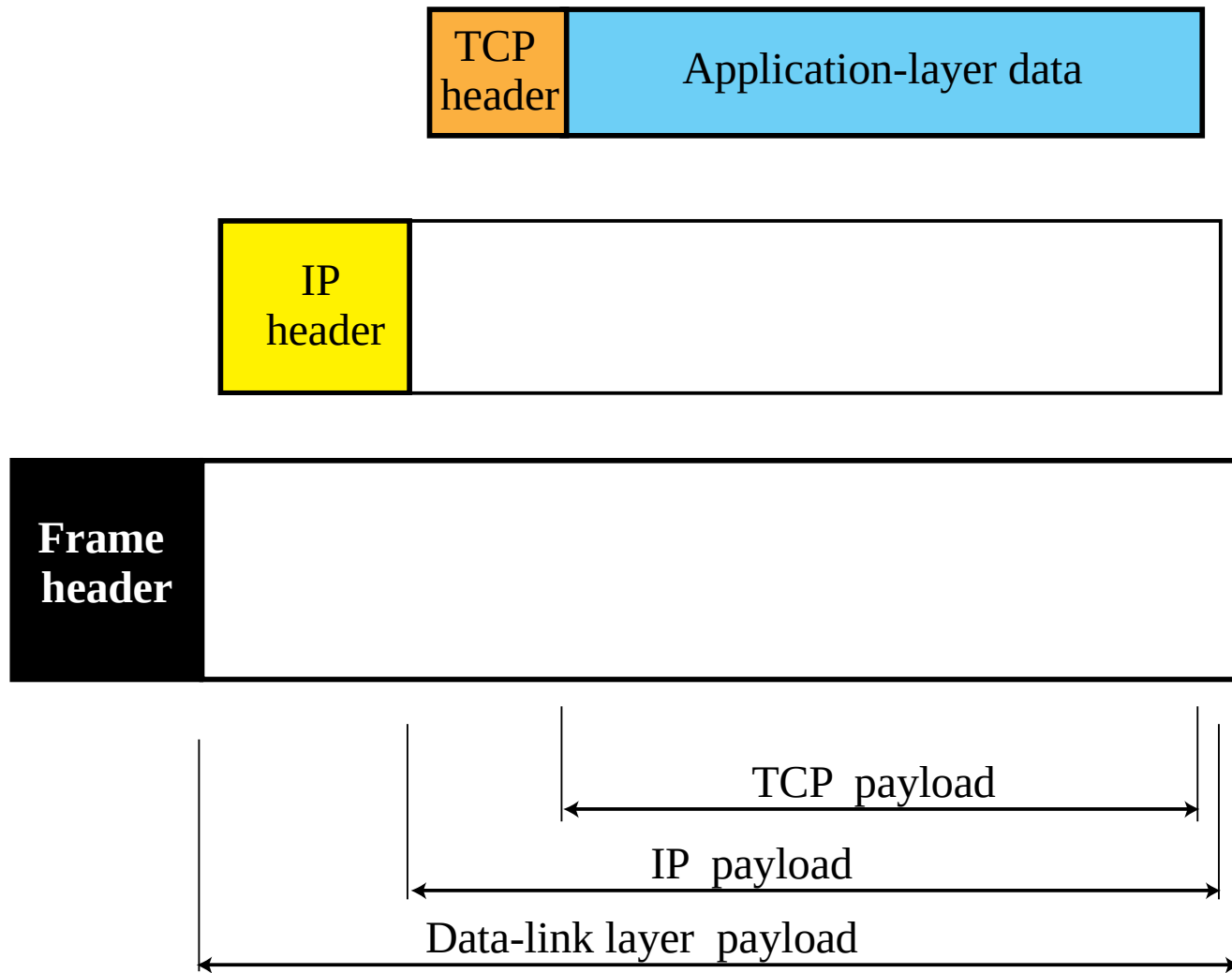
Figure 15.7 *Pseudoheader added to the TCP segment*



Note

The use of the checksum in TCP is mandatory.

Figure 15.8 *Encapsulation*



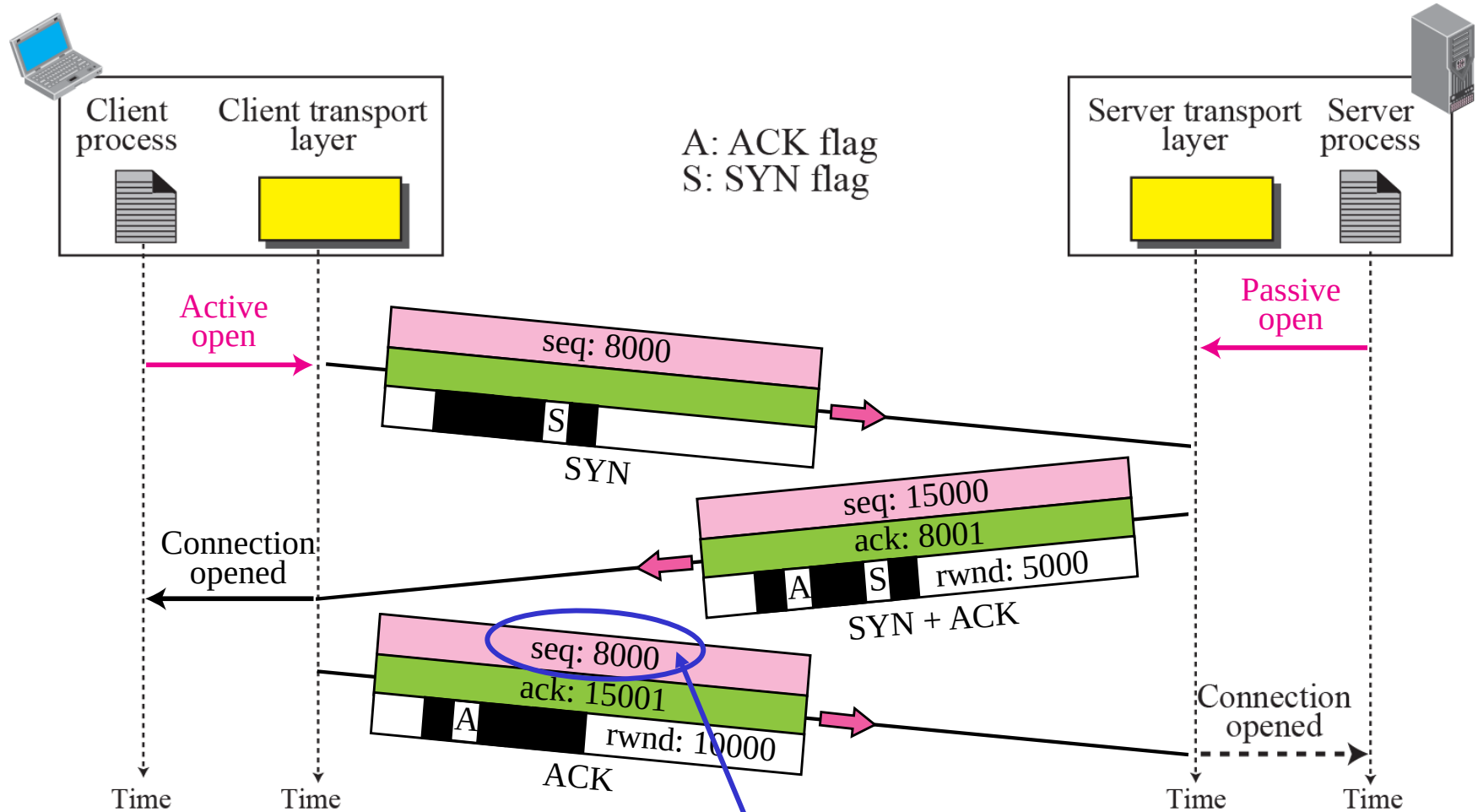
15-4 A TCP CONNECTION

TCP is connection-oriented. It establishes a virtual path between the source and destination. All of the segments belonging to a message are then sent over this virtual path. You may wonder how TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented. The point is that a TCP connection is virtual, not physical. TCP operates at a higher level. TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself. If a segment is lost or corrupted, it is retransmitted.

Topics Discussed in the Section

- ✓ **Connection Establishment**
- ✓ **Data Transfer**
- ✓ **Connection Termination**
- ✓ **Connection Reset**

Figure 15.9 Connection establishment using three-way handshake



Means "no data" !

seq: 8001 if piggybacking

Note

A SYN segment cannot carry data, but it consumes one sequence number.

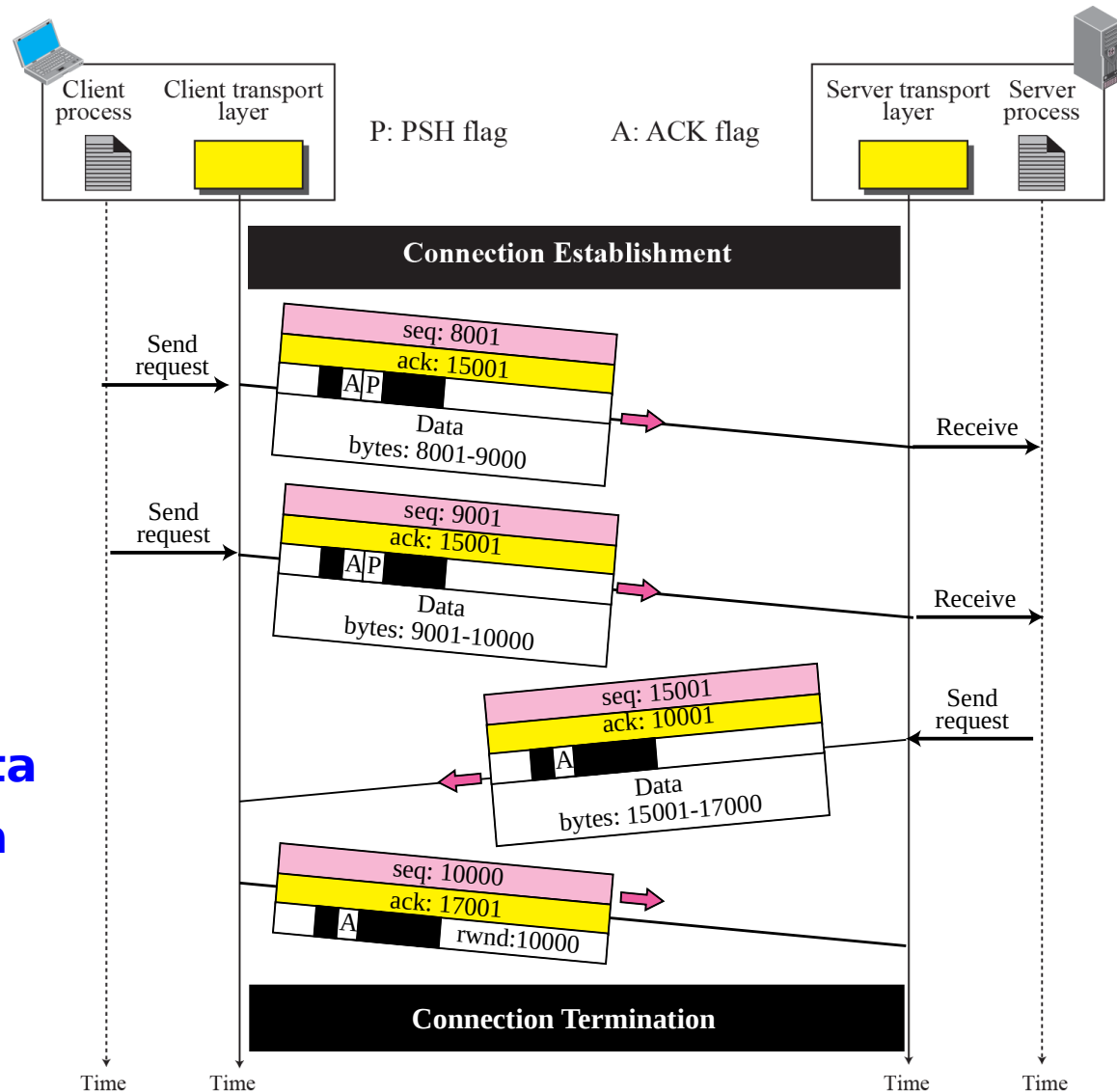
Note

A SYN + ACK segment cannot carry data, but does consume one sequence number.

Note

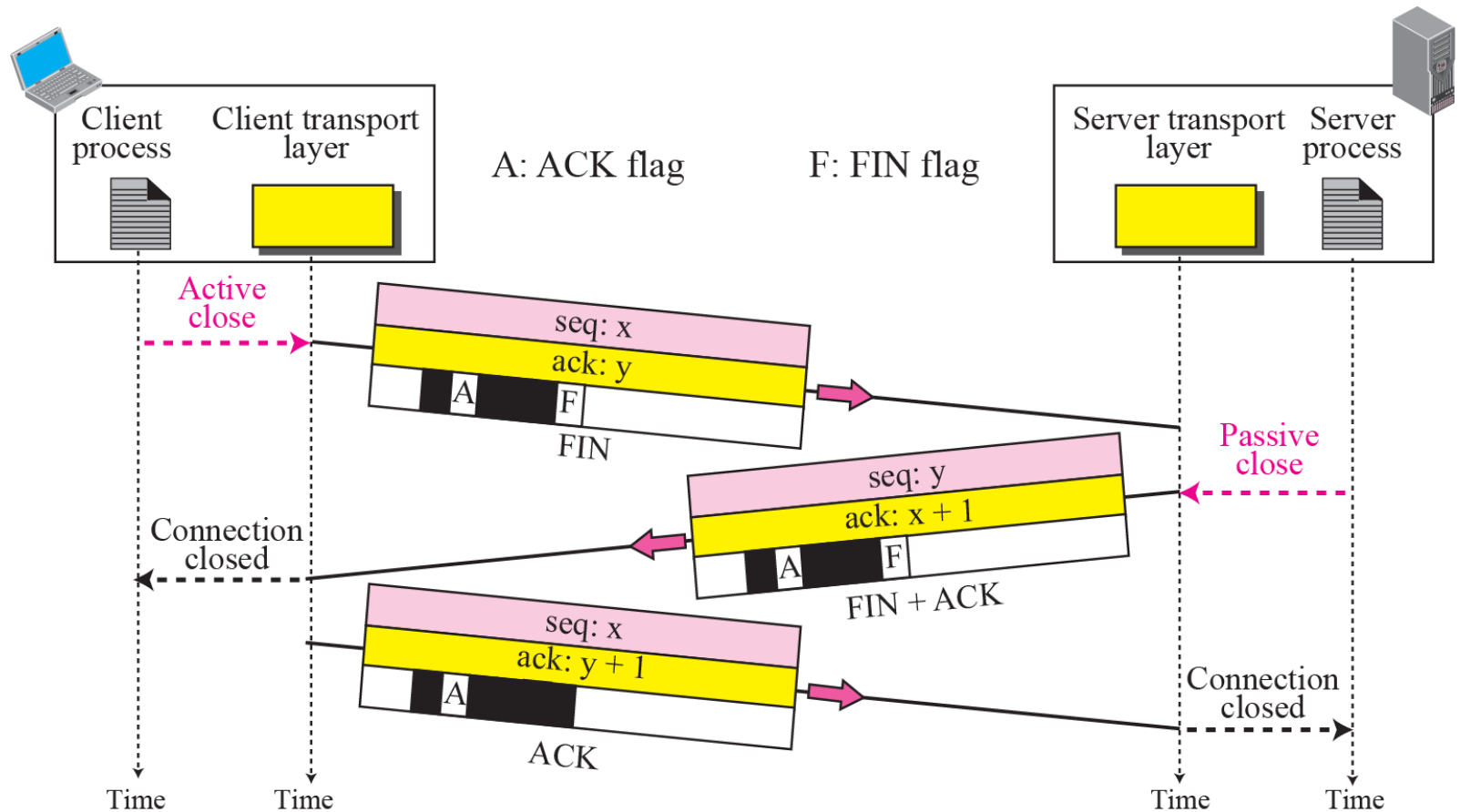
***An ACK segment, if carrying no data,
consumes no sequence number.***

Figure 15.10 *Data Transfer*



Pushing data
Urgent data

Figure 15.11 *Connection termination using three-way handshake*



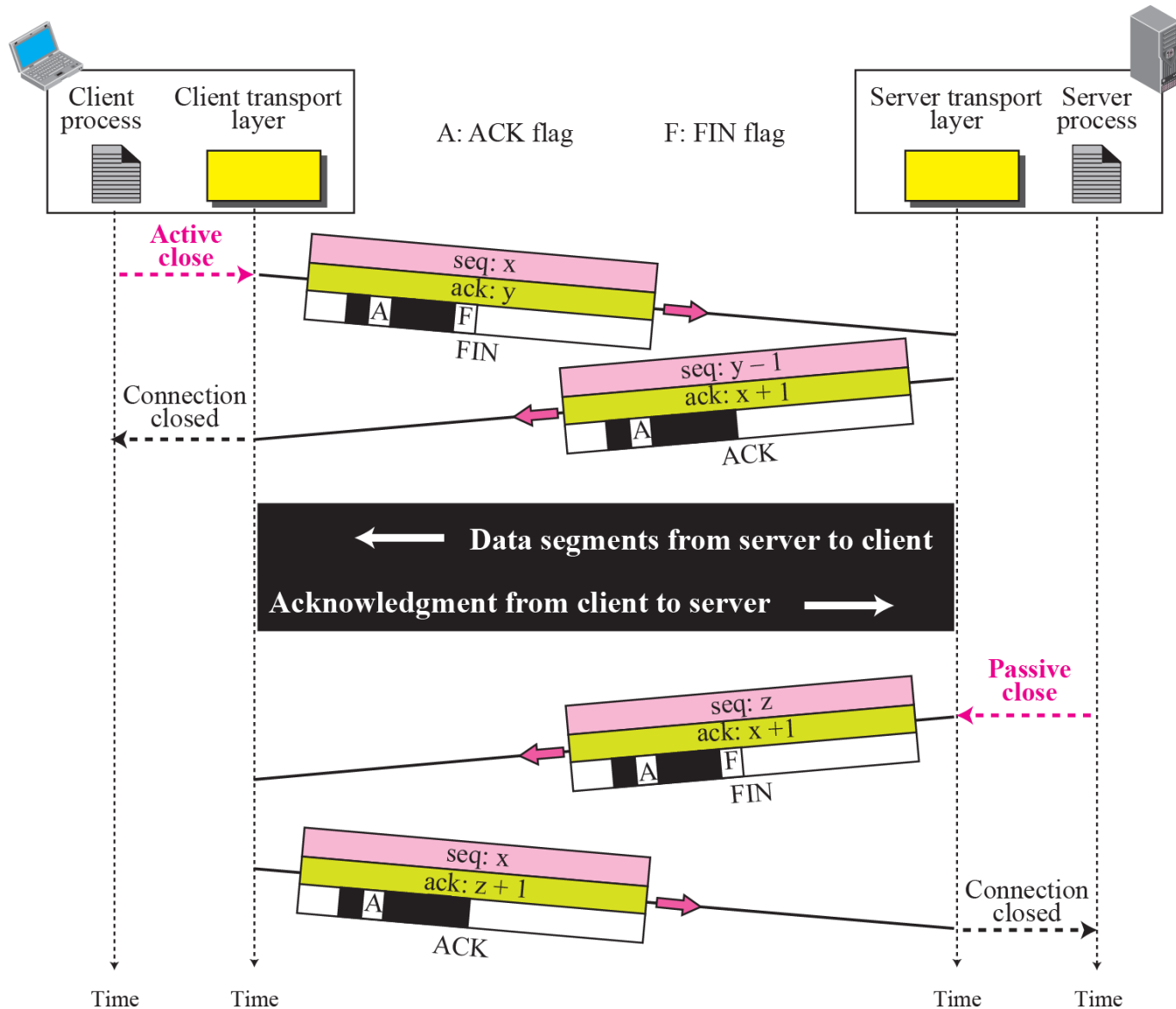
Note

The FIN segment consumes one sequence number if it does not carry data.

Note

The FIN + ACK segment consumes one sequence number if it does not carry data.

Figure 15.12 *Half-Close*



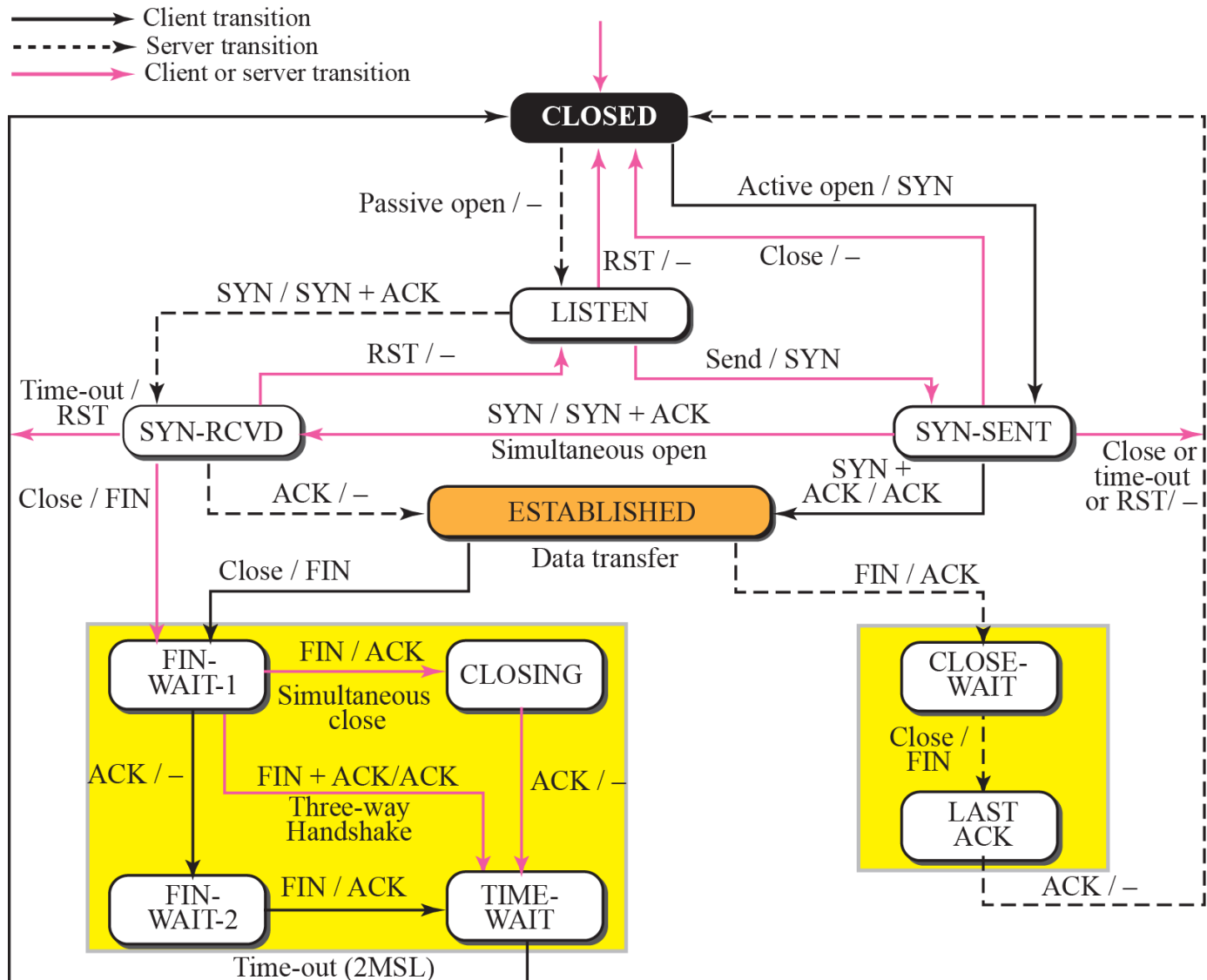
15-5 STATE TRANSITION DIAGRAM

To keep track of all the different events happening during connection establishment, connection termination, and data transfer, TCP is specified as the finite state machine shown in Figure 15.13.

Topics Discussed in the Section

✓ **Scenarios**

Figure 15.13 *State transition diagram*



Note

The state marked as ESTABLISHED in the FSM is in fact two different sets of states that the client and server undergo to transfer data.

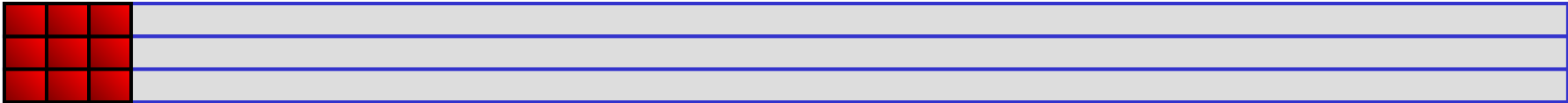


Table 15.2 *States for TCP*

<i>State</i>	<i>Description</i>
CLOSED	No connection exists
LISTEN	Passive open received; waiting for SYN
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN+ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer in progress
FIN-WAIT-1	First FIN sent; waiting for ACK
FIN-WAIT-2	ACK to first FIN received; waiting for second FIN
CLOSE-WAIT	First FIN received, ACK sent; waiting for application to close
TIME-WAIT	Second FIN received, ACK sent; waiting for 2MSL time-out
LAST-ACK	Second FIN sent; waiting for ACK
CLOSING	Both sides decided to close simultaneously

Figure 15.14 *Transition diagram for connection and half-close termination*

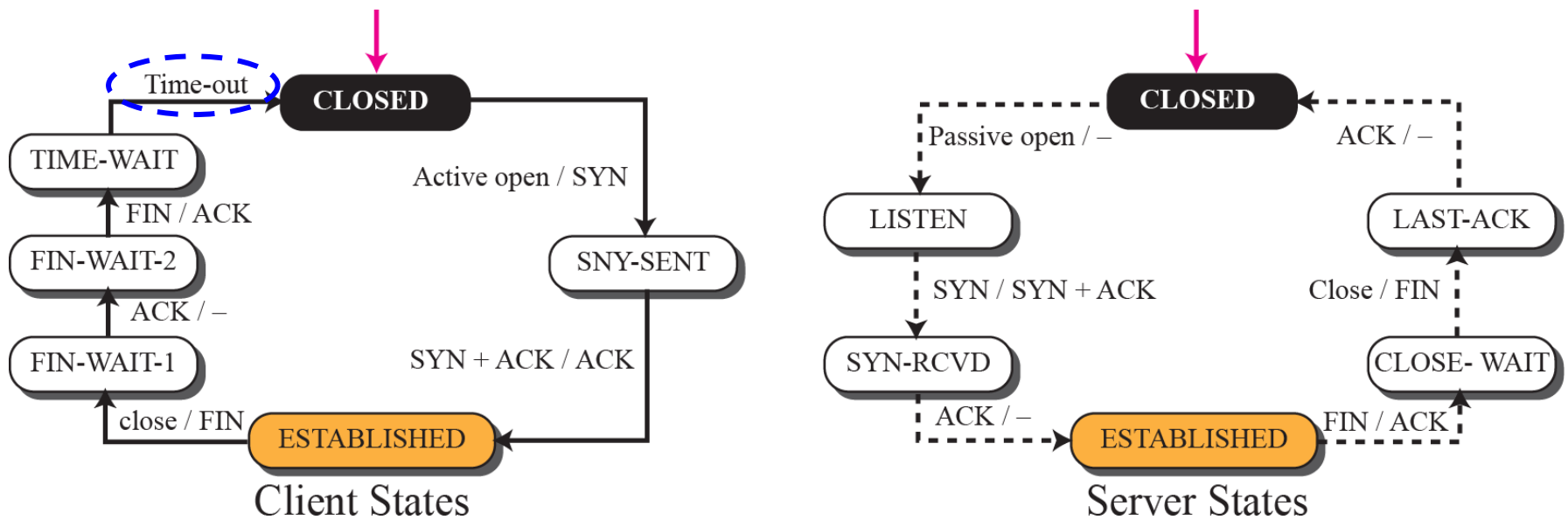


Figure 15.15 *Time-line diagram for Figure 15.14*

1. Enough time for an ACK to be lost and a new FIN to arrive. If during the TIME-WAIT state, a new FIN arrives, the client sends a new ACK and restarts the 2MSL timer
2. To prevent a duplicate segment from one connection appearing in the next one, TCP requires that incarnation cannot take place unless 2MSL amount of time has elapsed.

Another solution: the ISN of the incarnation is greater than the last seq. # used in the previous connection.

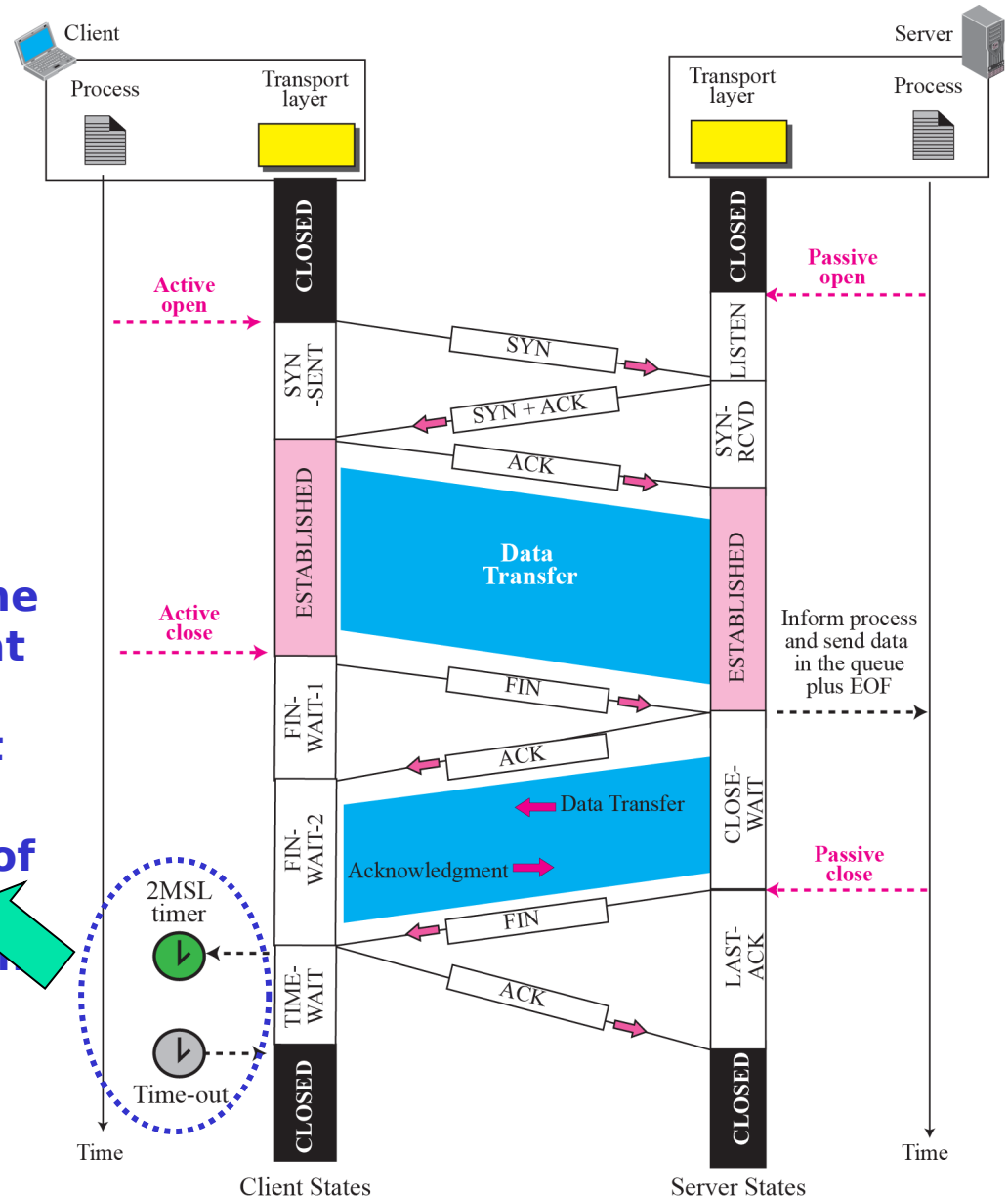


Figure 15.16 *Transition diagram for a common scenario*

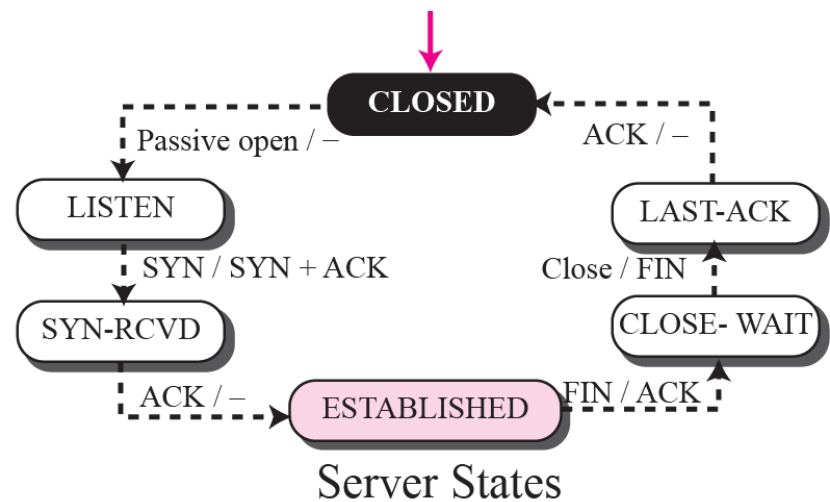
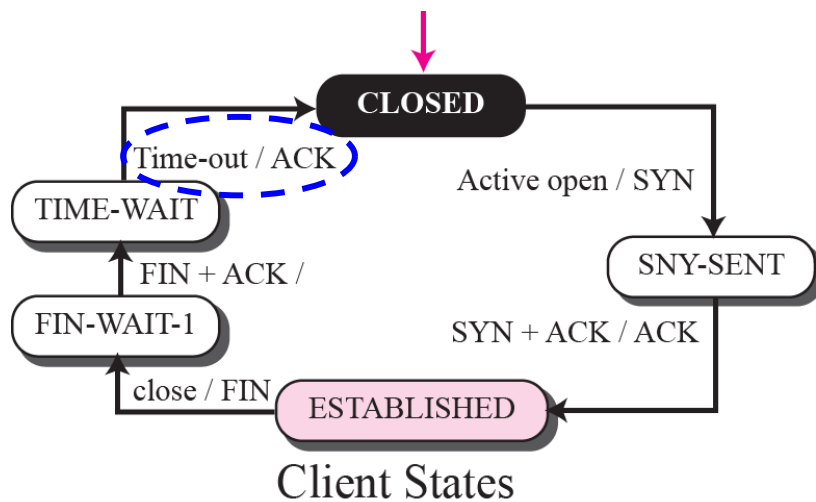


Figure 15.17 *Time line for a common scenario*

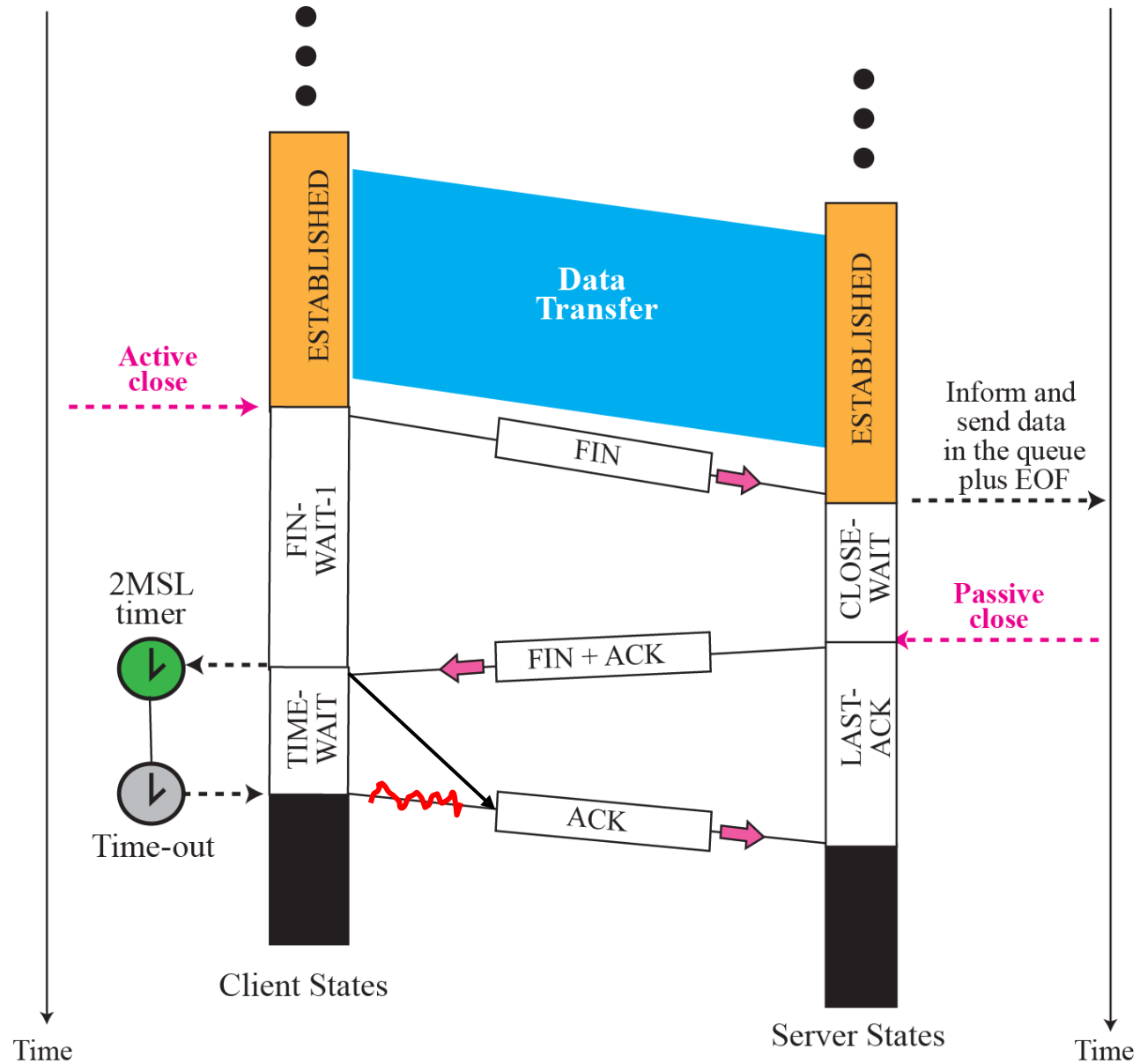


Figure 15.18 *Simultaneous open*

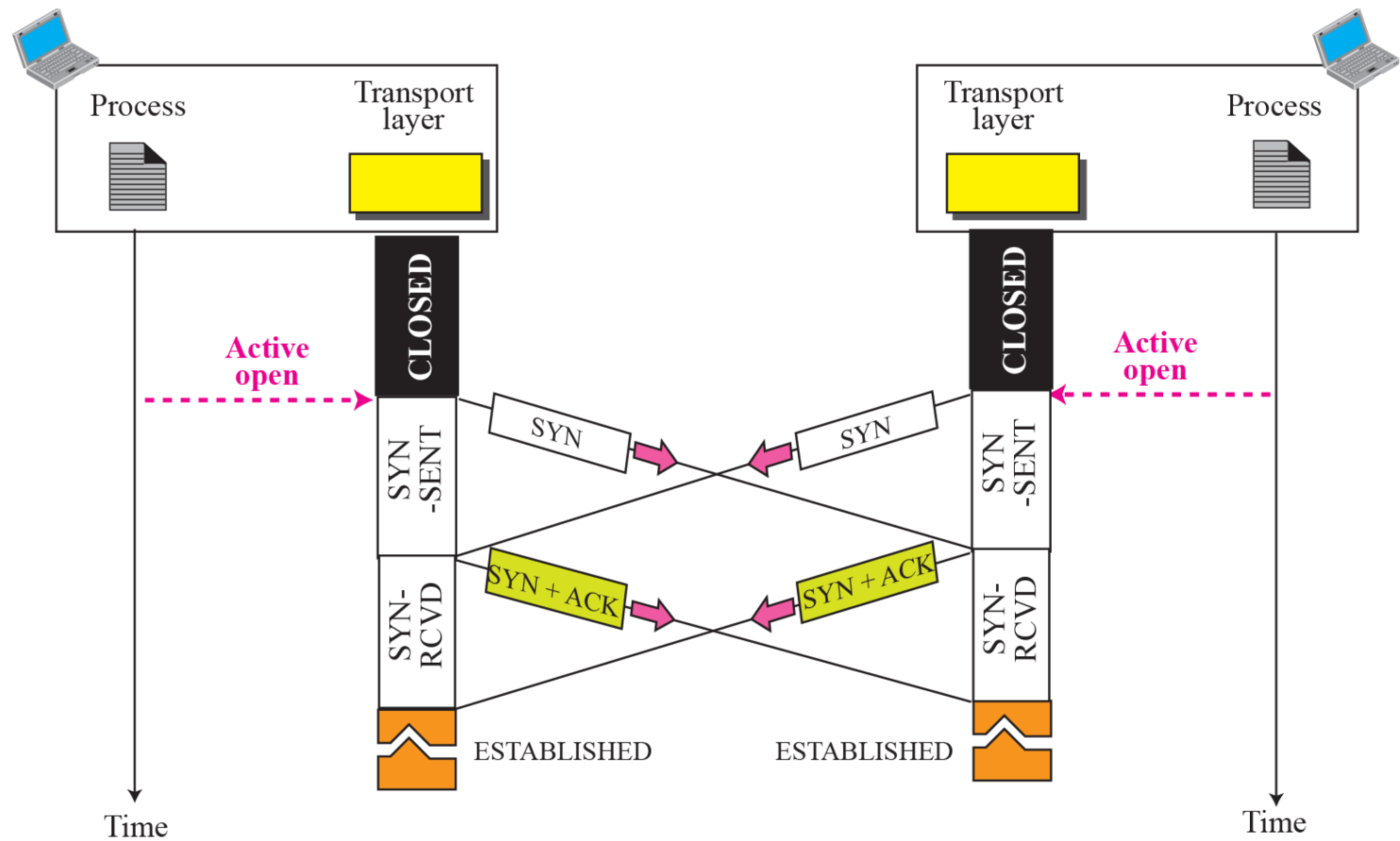


Figure 15.19 *Simultaneous close*

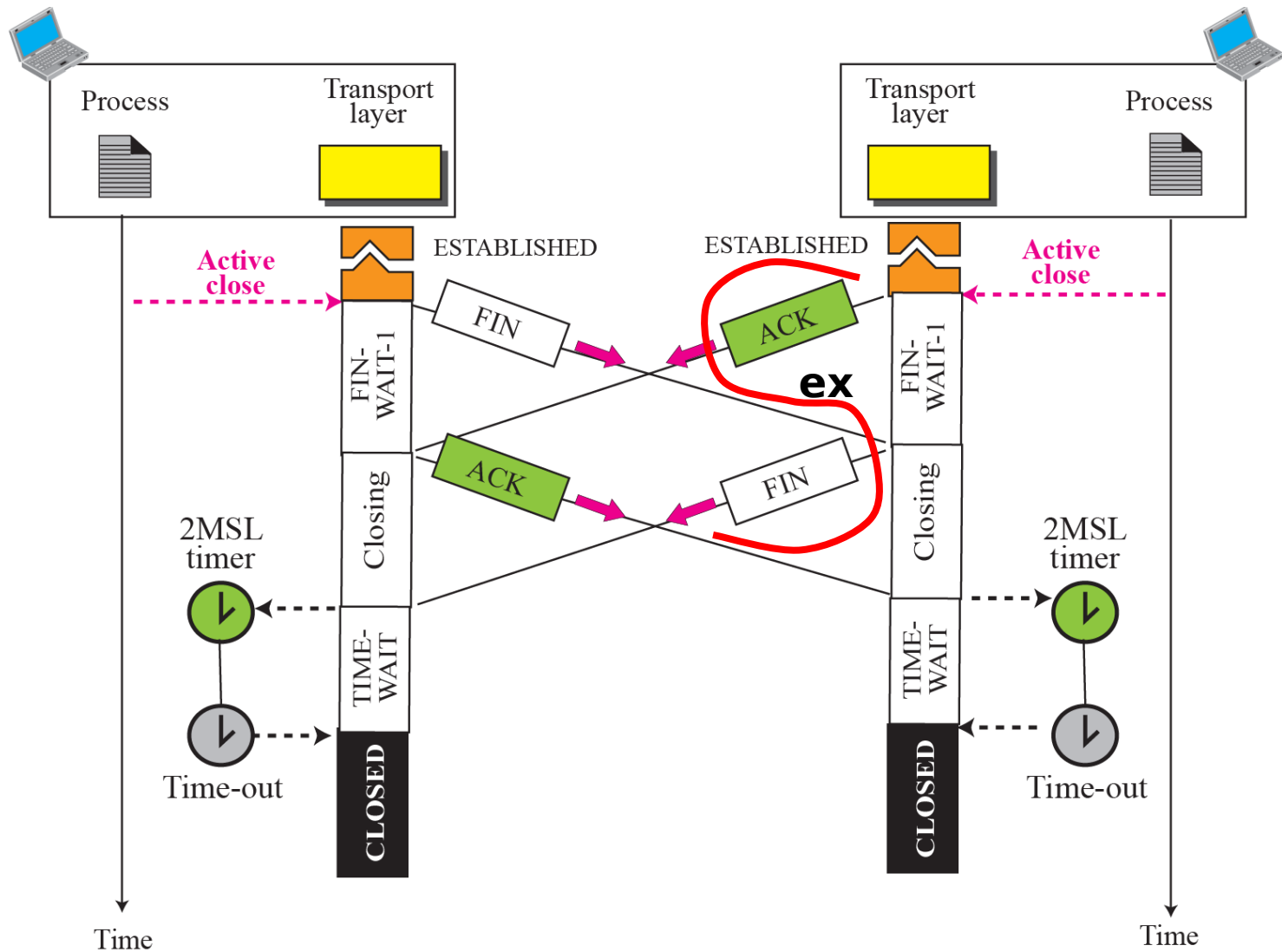


Figure 15.20 *Denying a connection*

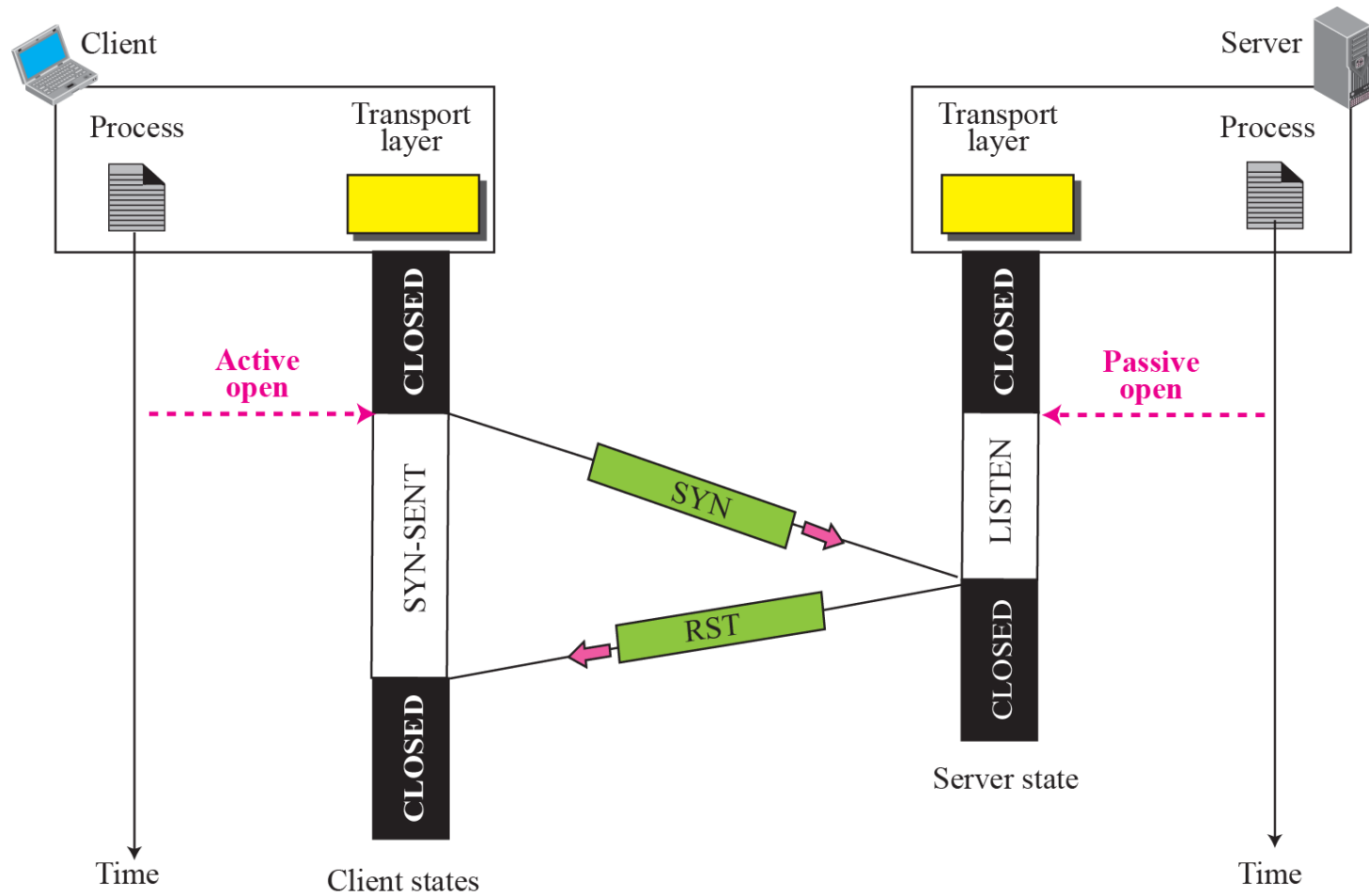
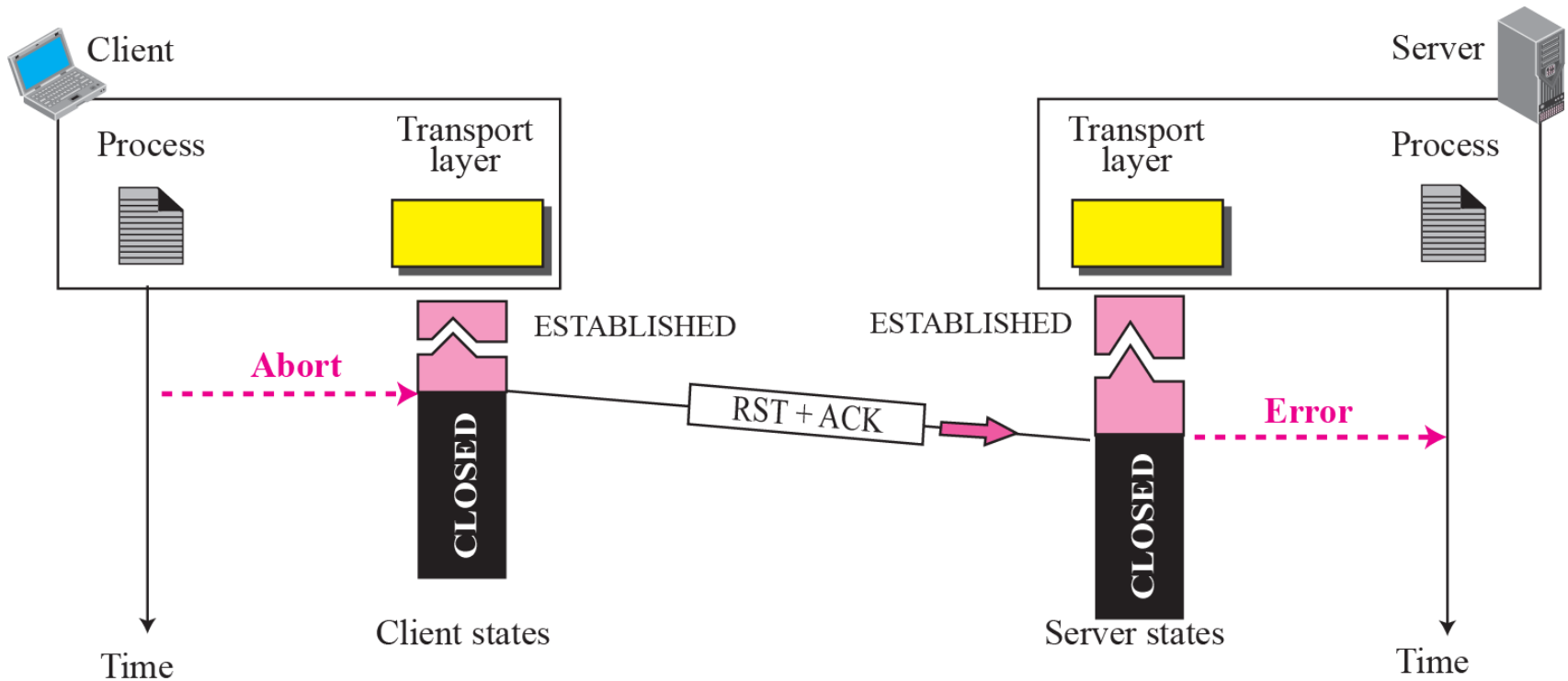


Figure 15.21 *Aborting a connection*



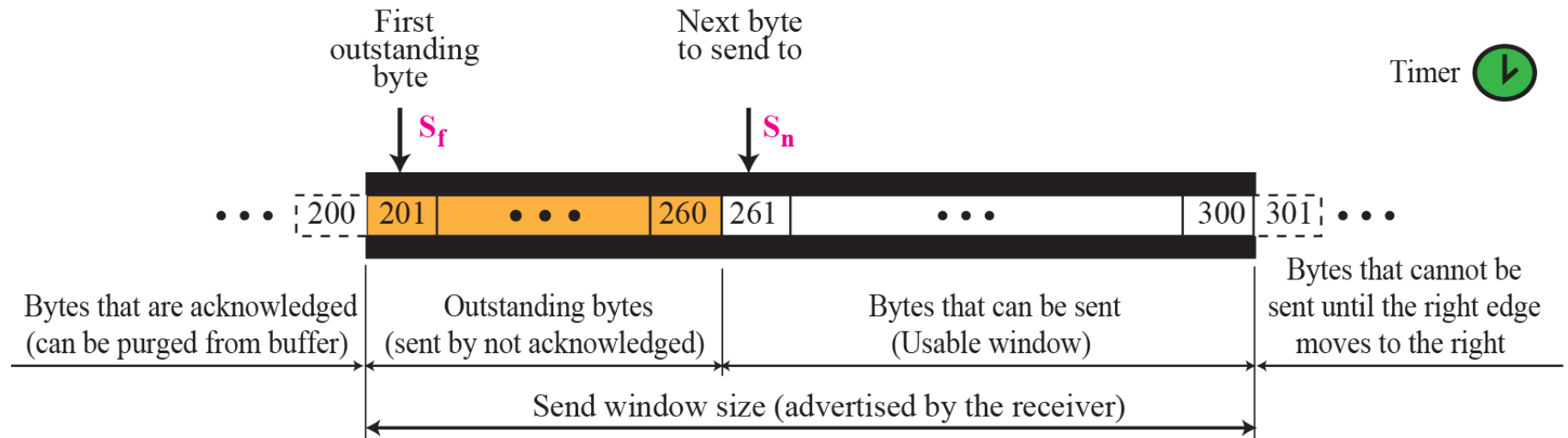
15-6 WINDOWS IN TCP

Before discussing data transfer in TCP and the issues such as flow, error, and congestion control, we describe the windows used in TCP. TCP uses two windows (send window and receive window) for each direction of data transfer, which means four windows for a bidirectional communication. To make the discussion simple, we make an assumption that communication is only unidirectional; the bidirectional communication can be inferred using two unidirectional communications with piggybacking.

Topics Discussed in the Section

- ✓ **Send Window**
- ✓ **Receive Window**

Figure 15.22 *Send window in TCP*

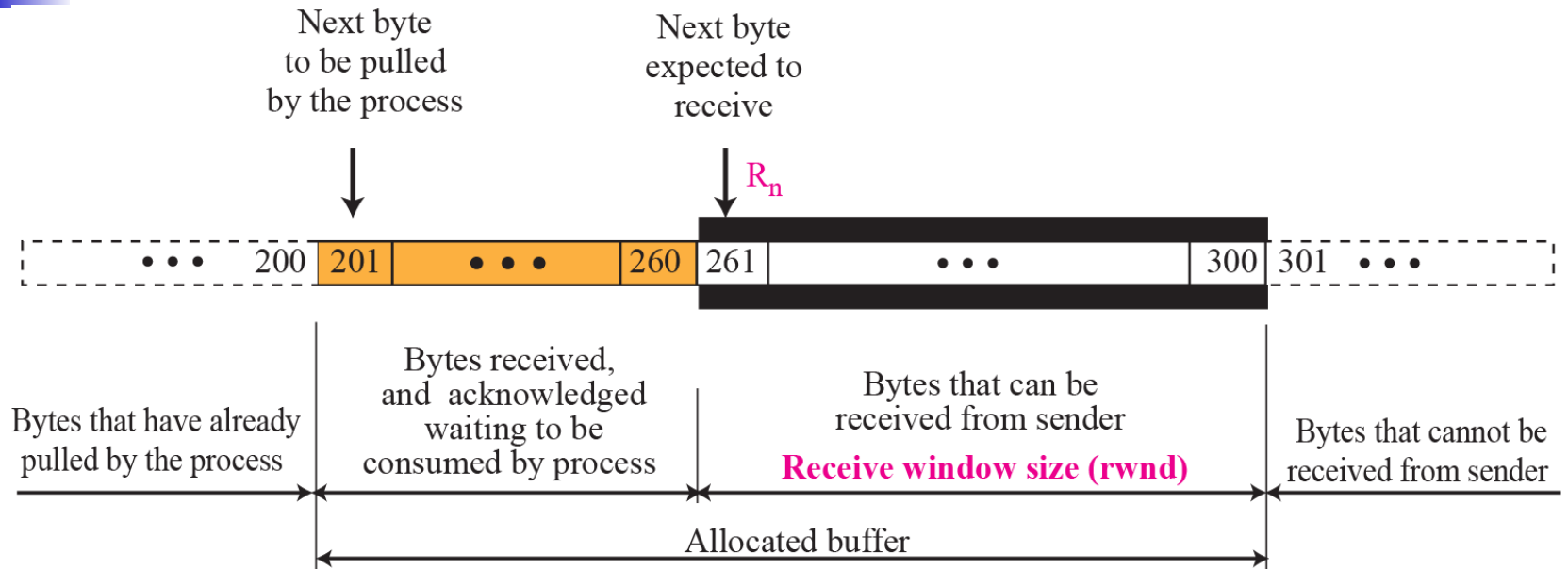


a. Send window

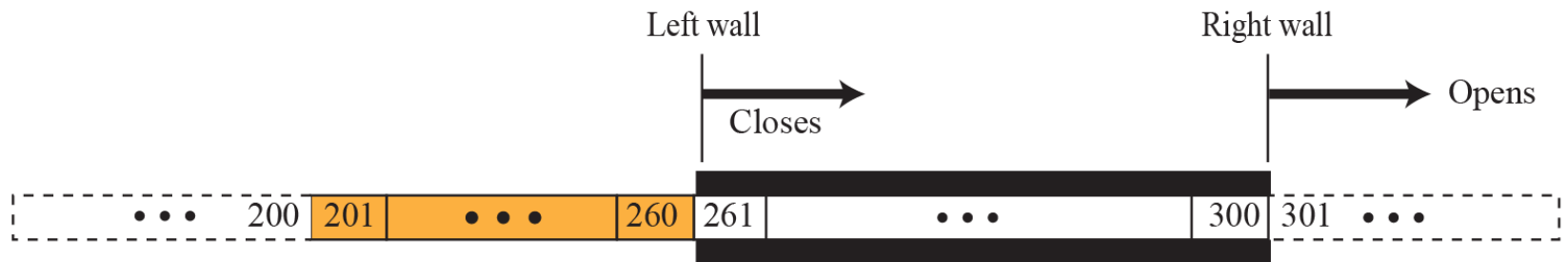


b. Opening, closing, and shrinking send window

Figure 15.23 *Receive window in TCP*



a. Receive window and allocated buffer



b. Opening and closing of receive window

15-7 FLOW CONTROL

As discussed in Chapter 13, flow control balances the rate a producer creates data with the rate a consumer can use the data. TCP separates flow control from error control. In this section we discuss flow control, ignoring error control. We temporarily assume that the logical channel between the sending and receiving TCP is error-free. Figure 15.24 shows unidirectional data transfer between a sender and a receiver; bidirectional data transfer can be deduced from unidirectional one as discussed in Chapter 13.

Topics Discussed in the Section

- ✓ **Opening and Closing Windows**
- ✓ **Shrinking of Windows**
- ✓ **Silly Window Syndrome**

Figure 15.24 *TCP/IP protocol suite*

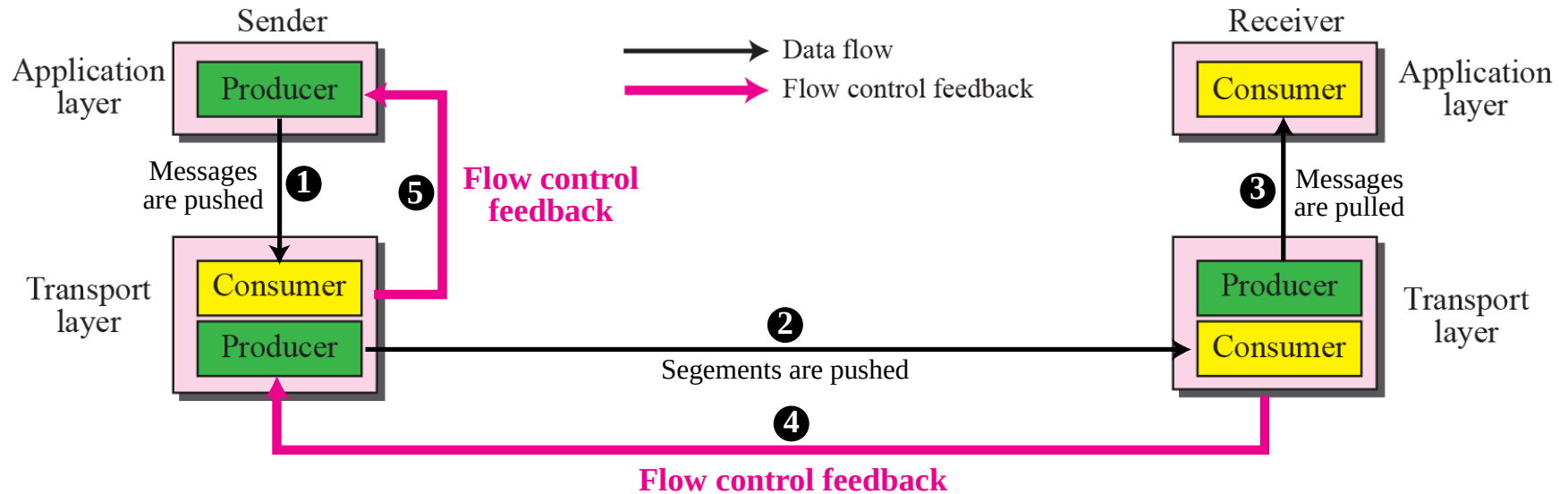
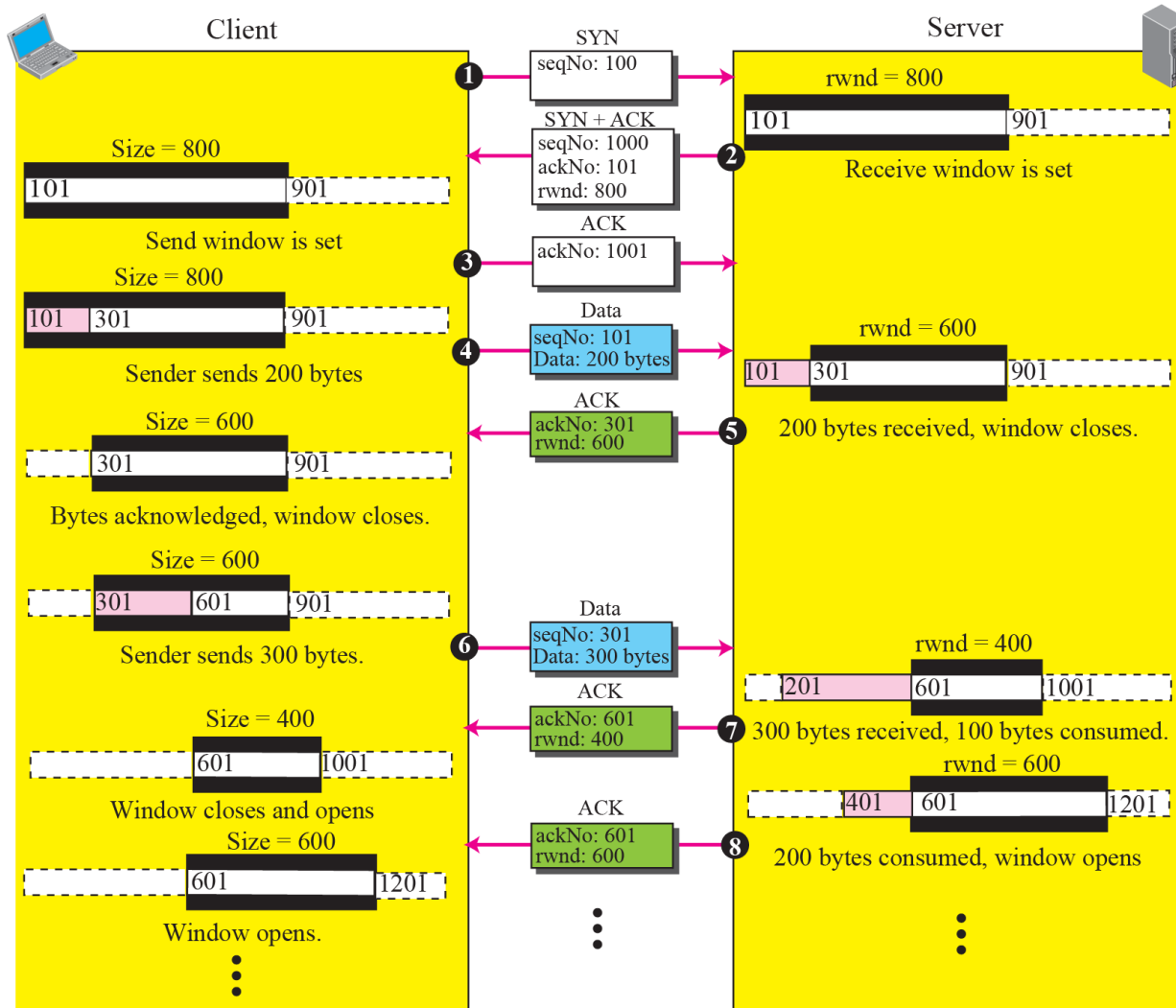


Figure 15.25 *An example of flow control*

Note: We assume only unidirectional communication from client to server. Therefore, only one window at each side is shown.

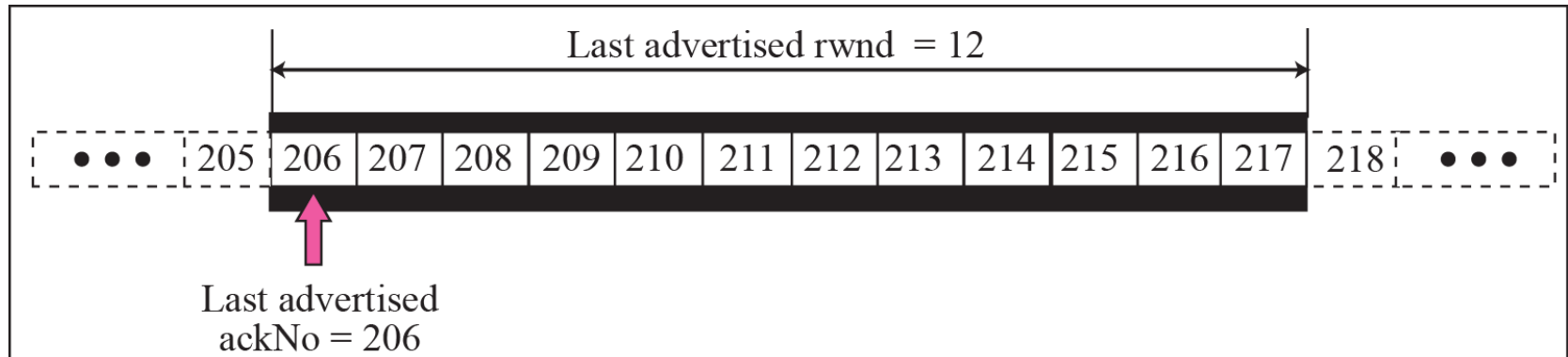


Example 15.2

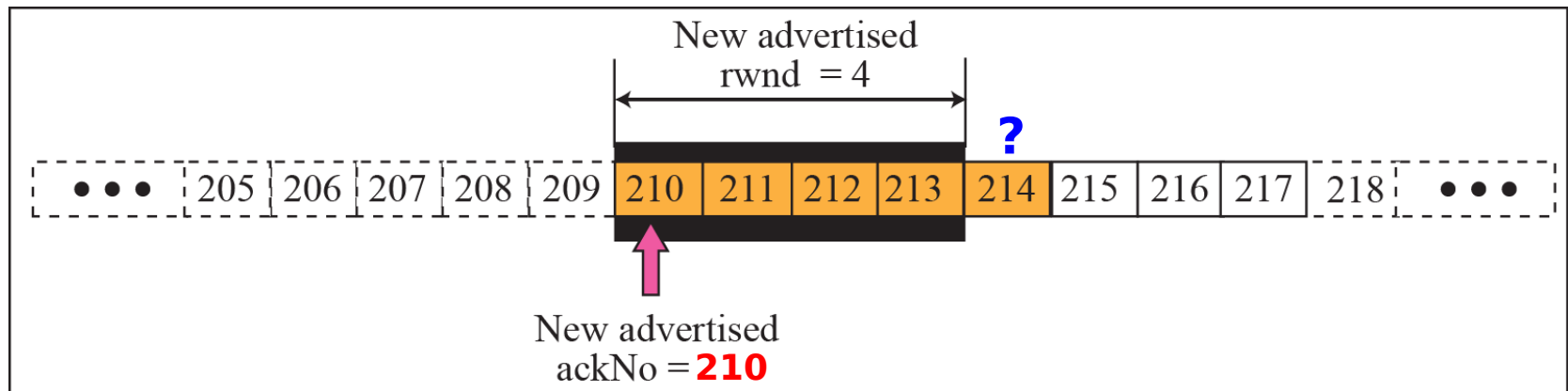
Figure 15.26 shows the reason for the mandate in window shrinking. Part a of the figure shows values of last acknowledgment and *rwnd*. Part b shows the situation in which the sender has sent bytes 206 to 214. Bytes 206 to 209 are acknowledged and purged. The new advertisement, however, defines the new value of *rwnd* as 4, in which $210 + 4 < 206 + 12$. When the send window shrinks, it creates a problem: byte 214 which has been already sent is outside the window. The relation discussed before forces the receiver to maintain the right-hand wall of the window to be as shown in part a because the receiver does not know which of the bytes 210 to 217 has already been sent. One way to prevent this situation is to let the receiver postpone its feedback until enough buffer locations are available in its window. In other words, the receiver should wait until more bytes are consumed by its process.

Figure 15.26 Example 15.2

Prevent the shrinking of the send window:
 $\text{new ackNo} + \text{new rwnd} \geq \text{last ackNo} + \text{last rwnd}$

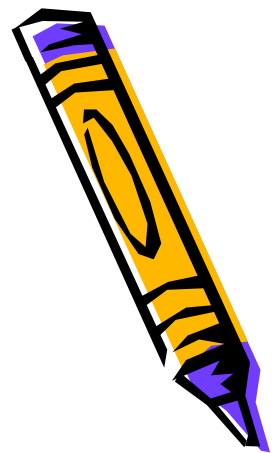


a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

Silly Window Syndrome (1)



➤ *Sending data in very small segments*

1. Syndrome created by the Sender

- *Sending application program creates data slowly (e.g. 1 byte at a time)*
- *Wait and collect data to send in a larger block*
- *How long should the sending TCP wait?*
- *Solution: **Nagle's algorithm***
- *Nagle's algorithm takes into account (1) **the speed of the application program** that creates the data, and (2) **the speed of the network** that transports the data*



Silly Window Syndrome (2)



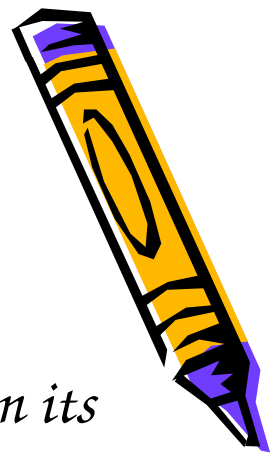
2. Syndrome created by the Receiver

- Receiving application program consumes data slowly (e.g. 1 byte at a time)*
- The receiving TCP announces a window size of 1 byte. The sending TCP sends only 1 byte...*
- Solution 1: **Clark's solution***
- Sending an ACK but announcing a window size of zero until there is enough space to accommodate a segment of max. size or until half of the buffer is empty*



Silly Window Syndrome (3)

- Solution 2: *Delayed Acknowledgement*
- *The receiver waits until there is decent amount of space in its incoming buffer before acknowledging the arrived segments*
- *The delayed acknowledgement prevents the sending TCP from sliding its window. It also reduces traffic.*
- *Disadvantage: it may force the sender to retransmit the unacknowledged segments*
- *To balance: should not be delayed by more than 500ms*



15-8 ERROR CONTROL

TCP is a reliable transport layer protocol. This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.

Error control in TCP is achieved through the use of three tools:
checksum, acknowledgment, and time-out

Topics Discussed in the Section

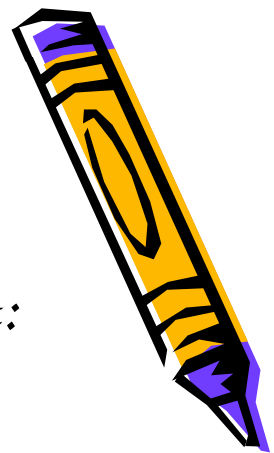
- ✓ **Checksum**
- ✓ **Acknowledgment**
- ✓ **Retransmission**
- ✓ **Out-of-Order Segments**
- ✓ **FSMs for Data Transfer in TCP**
- ✓ **Some Scenarios**

Note

***ACK segments do not consume
sequence numbers and
are not acknowledged.***

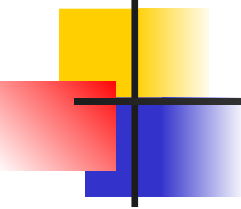
Acknowledgement Type

- In the past, TCP used only one type of acknowledgement: Accumulative Acknowledgement (**ACK**), also namely *accumulative positive acknowledgement*
- More and more implementations are adding another type of acknowledgement: *Selective Acknowledgement (SACK)*, SACK is implemented as an option at the end of the TCP header.



Note

Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order data are delivered to the process.



Note

*TCP can be best modeled as a
Selective Repeat protocol.*

Figure 15.27 *Simplified FSM for sender site*

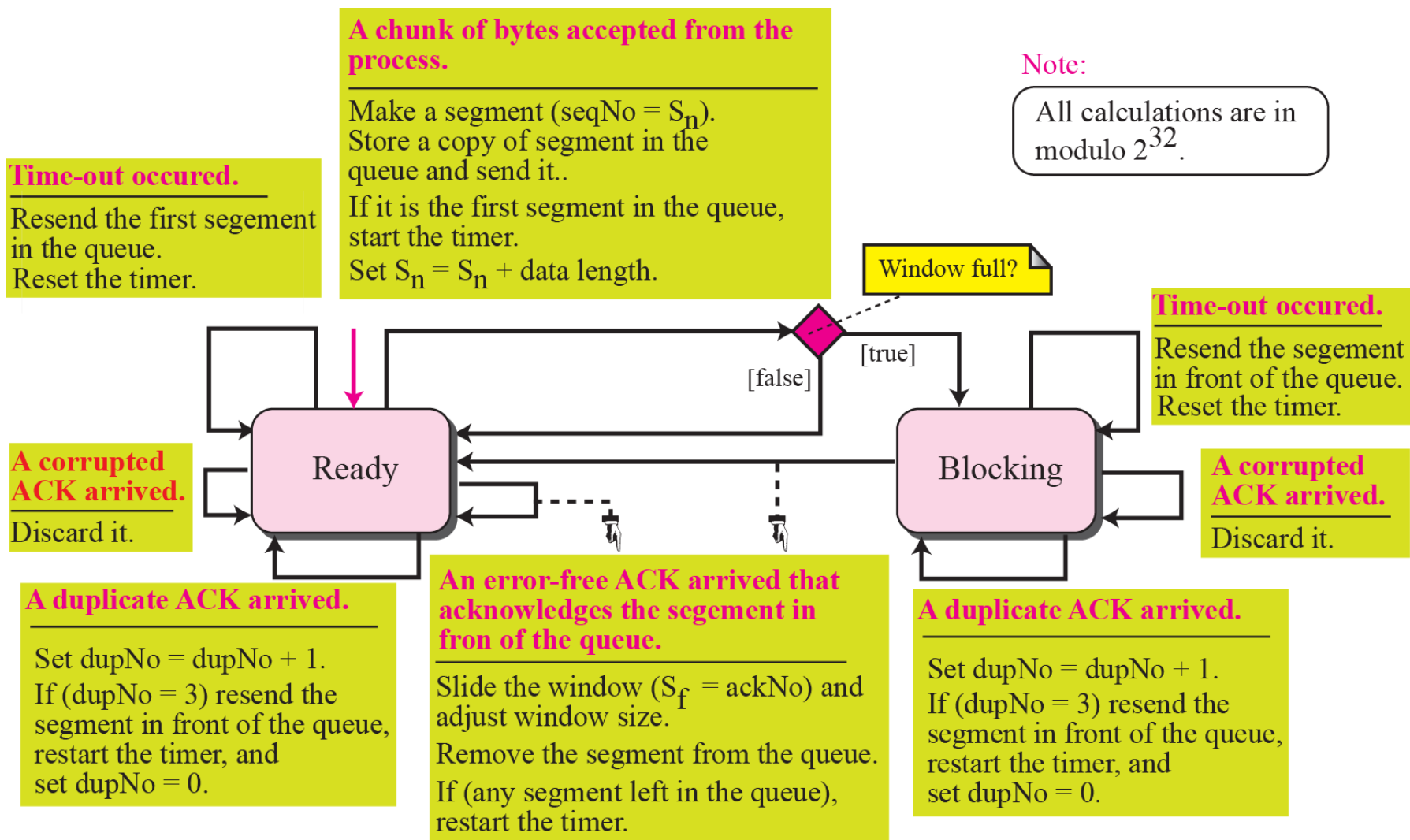


Figure 15.28 *Simplified FSM for the receiver site*

Note:

All calculations are in modulo 2^{32} .

An expected error-free segment arrived.

Buffer the message.

$R_n = R_n + \text{data length}$.

If the ACK-delaying timer is running, stop the timer and send a cumulative ACK. Else, start the ACK-delaying timer.

A request for delivery of k bytes of data from process came

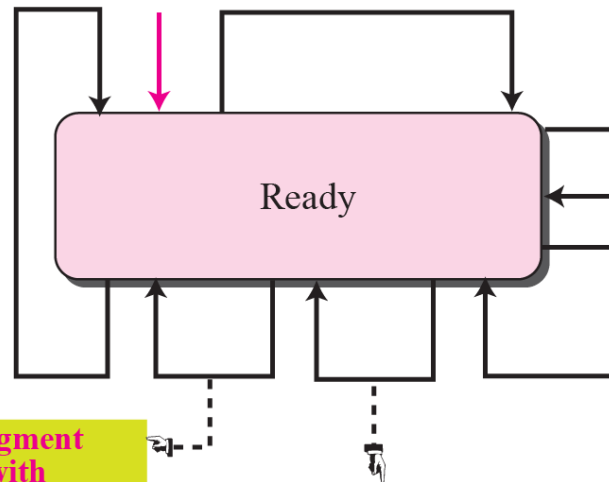
Deliver the data.

Slide the window and adjust window size.

An error-free duplicate segment or an error-free segment with sequence number outside window arrived

Discard the segment.

Send an ACK with ackNo equal to the sequence number of expected segment (duplicate ACK).



ACK-delaying timer expired.

Send the delayed ACK.

An error-free, but out-of-order segment arrived

Store the segment if not duplicate.

Send an ACK with ackNo equal to the sequence number of expected segment (duplicate ACK).

A corrupted segment arrived

Discard the segment.

Rules for Generating ACK(1)

- 1. When one end sends a data segment to the other end, it must include an ACK. That gives the next sequence number it expects to receive. (*Piggyback*)
- 2. The receiver needs to *delay sending* (until *another segment arrives or 500ms*) an ACK segment if there is only one outstanding in-order segment. It prevents ACK segments from creating extra traffic.
- 3. There *should not be more than 2 in-order unacknowledged segments* at any time. It prevent the unnecessary retransmission



Rules for Generating ACK(2)

- 4. When a *segment arrives with an out-of-order sequence number* that is higher than expected, the receiver immediately sends an ACK segment announcing the sequence number of the next expected segment. (for *fast retransmission*)
- 5. When a *missing segment* arrives, the receiver sends an ACK segment to announce the next sequence number expected.
- 6. If a *duplicate segment* arrives, the receiver immediately sends an ACK.



Figure 15.29 *Normal operation*

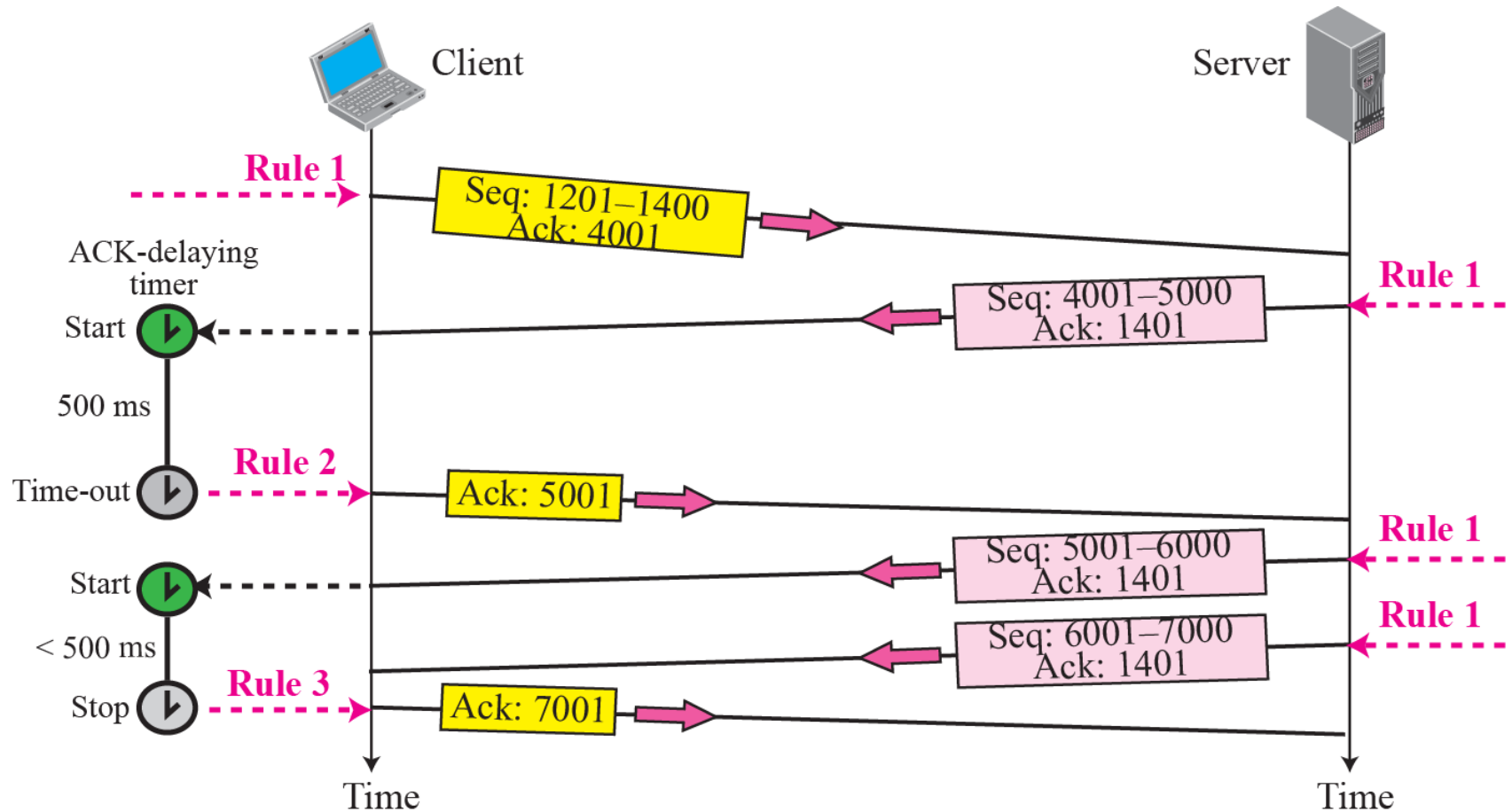
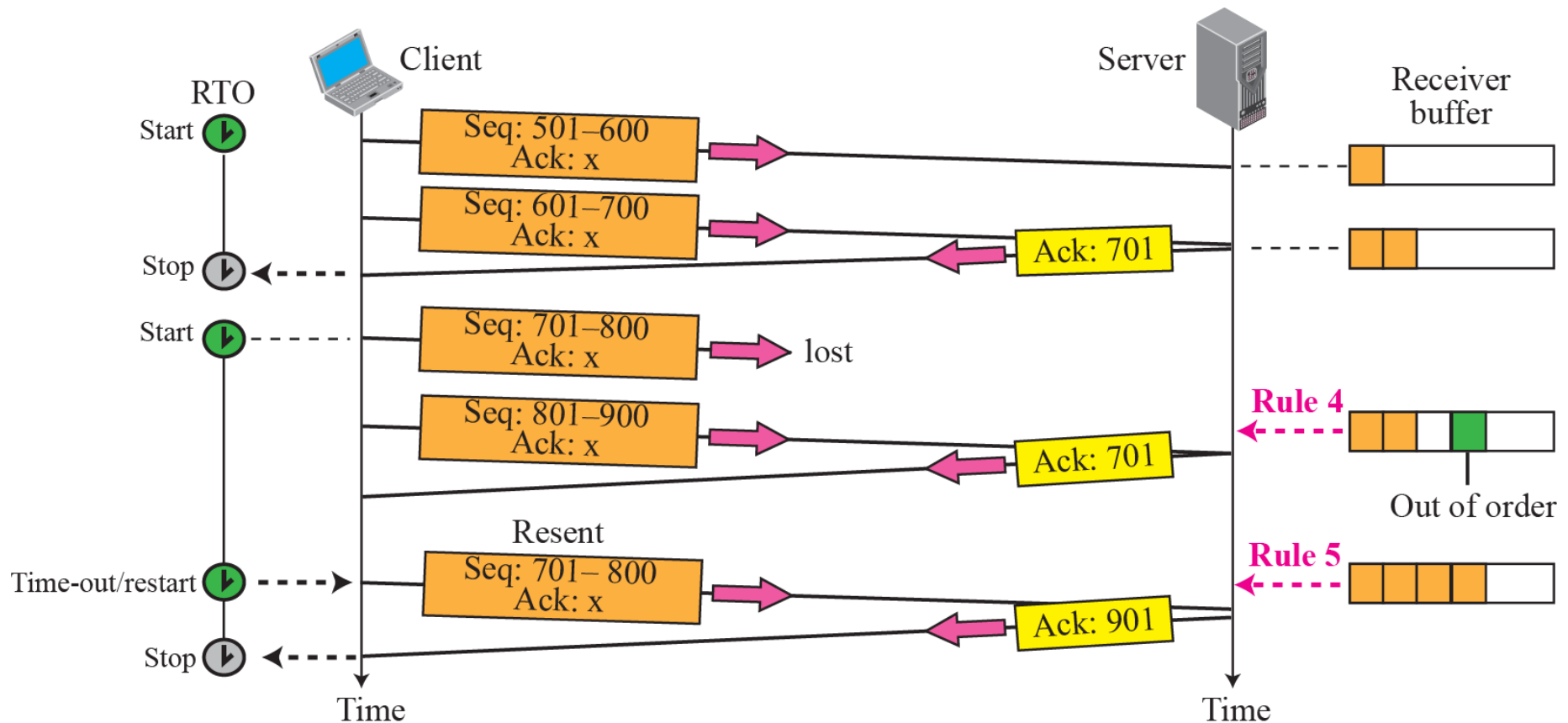


Figure 15.30 *Lost segment*



Note

The receiver TCP delivers only ordered data to the process.

Figure 15.31 *Fast retransmission*

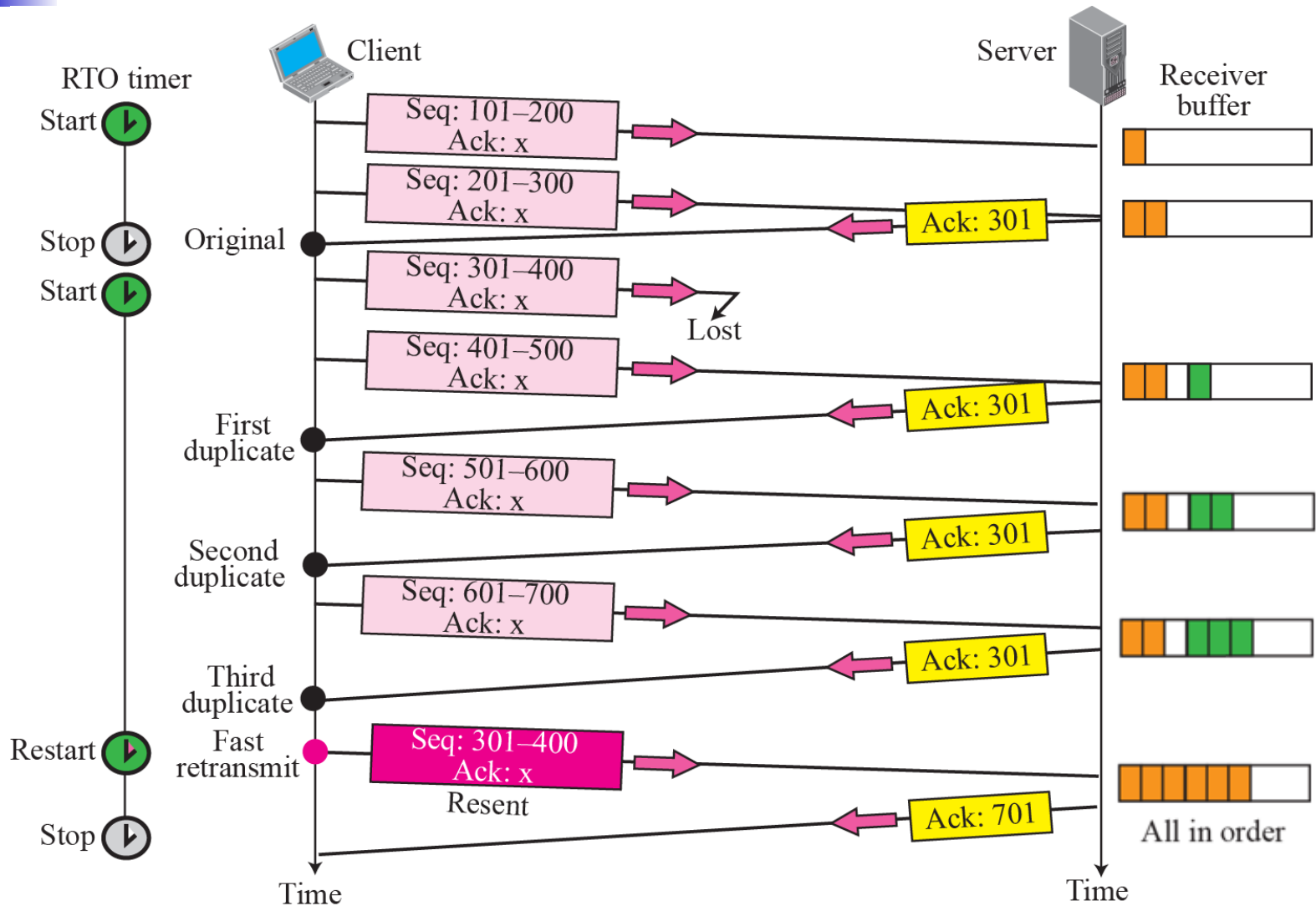


Figure 15.32 *Lost acknowledgment*

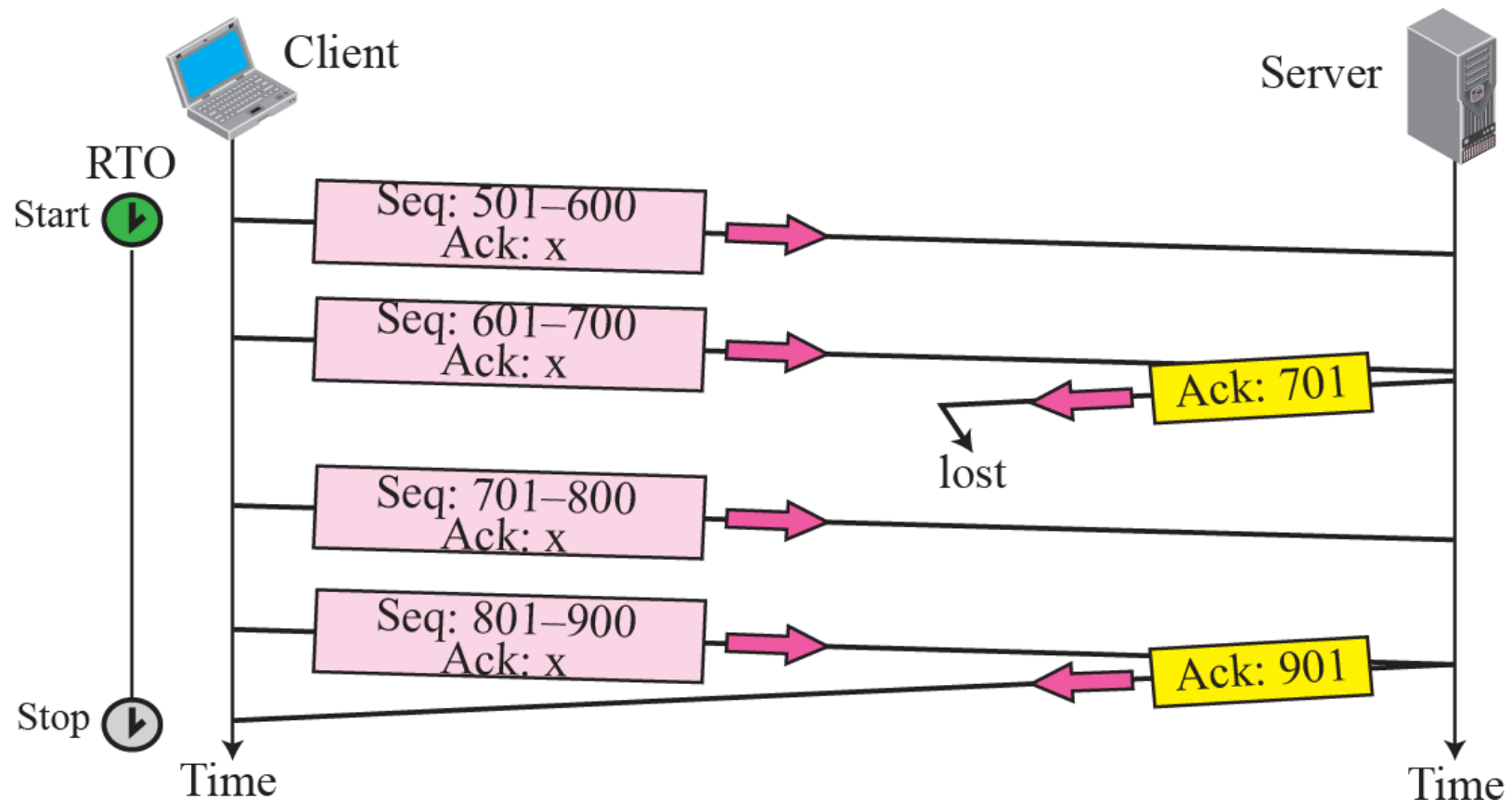
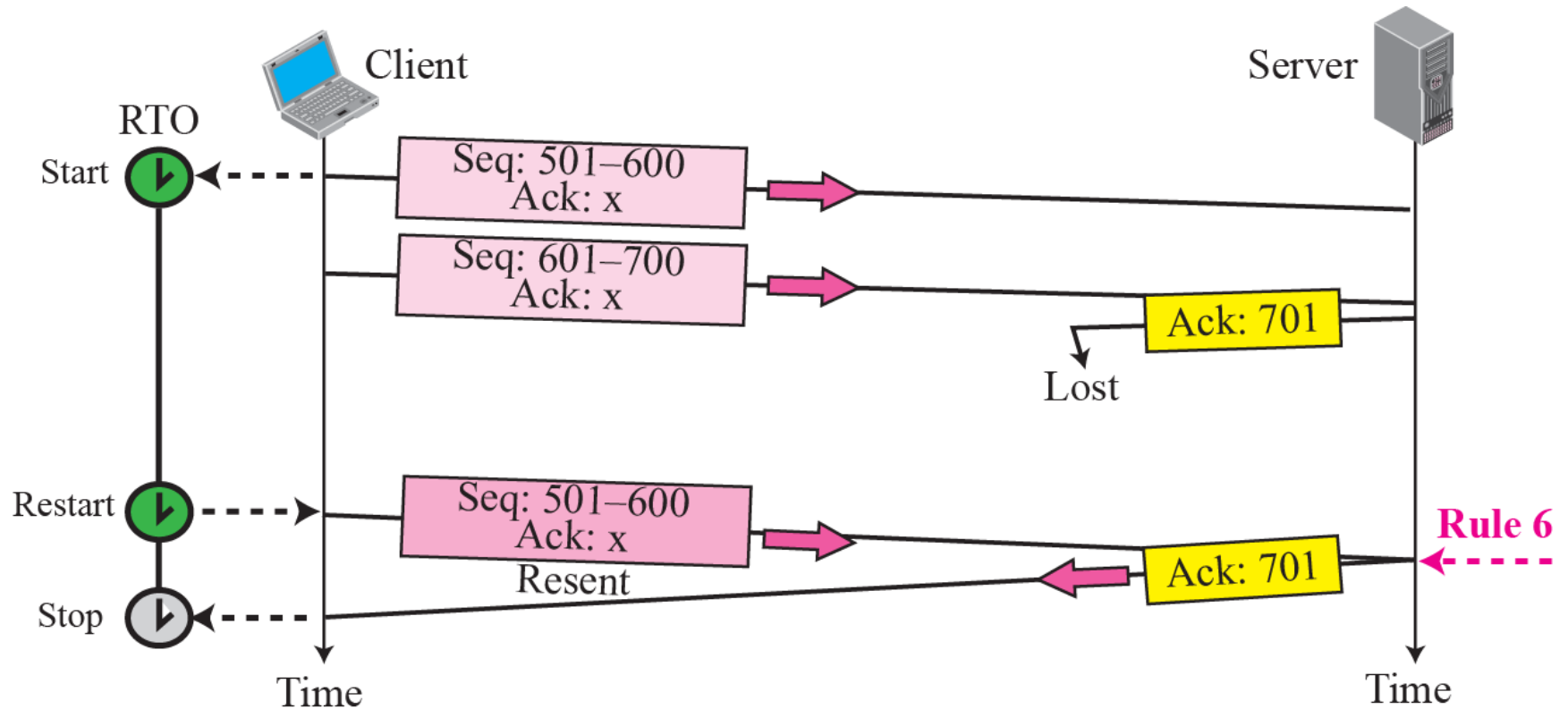


Figure 15.33 *Lost acknowledgment corrected by resending a segment*



Note

***Lost acknowledgments may create
deadlock if they are not
properly handled.***

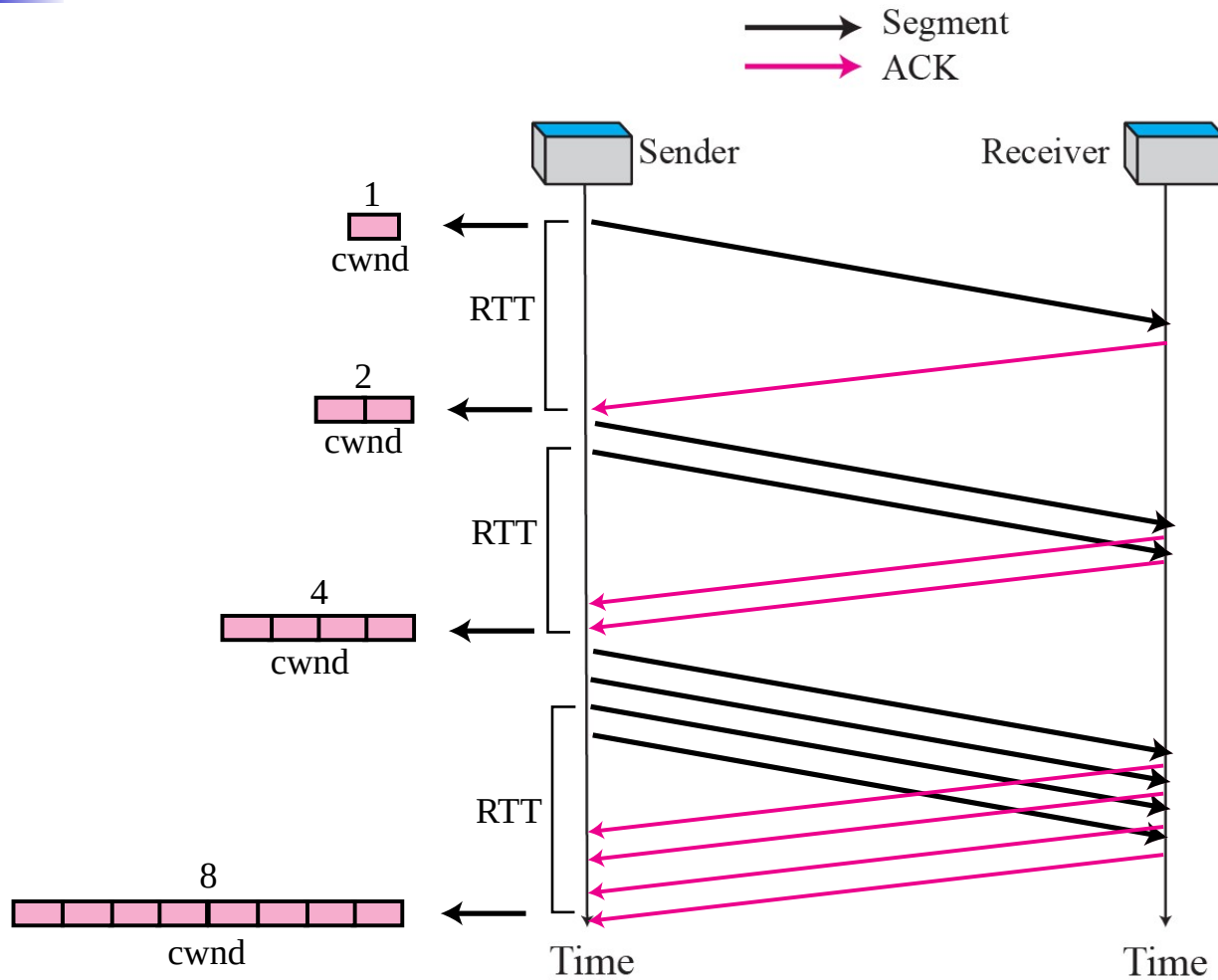
15-9 CONGESTION CONTROL

We discussed congestion control in Chapter 13. Congestion control in TCP is based on both open loop and closed-loop mechanisms. TCP uses a congestion window and a congestion policy that avoid congestion and detect and alleviate congestion after it has occurred.

Topics Discussed in the Section

- ✓ **Congestion Window**
- ✓ **Congestion Policy**

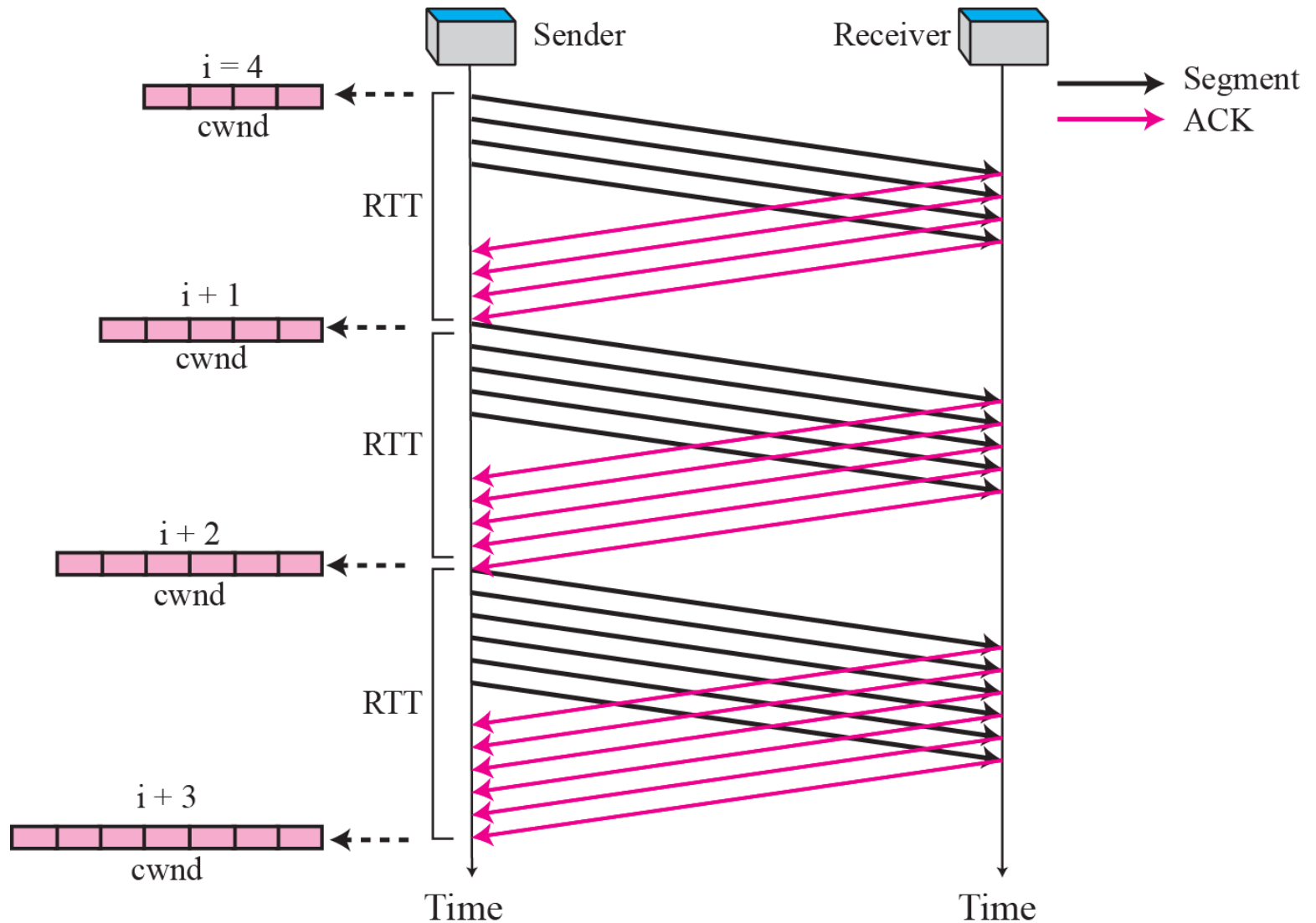
Figure 15.34 *Slow start, exponential increase*



Note

In the slow start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

Figure 15.35 *Congestion avoidance, additive increase*



Note

In the congestion avoidance algorithm the size of the congestion window increases additively until congestion is detected.

Figure 15.36 *TCP Congestion policy summary*

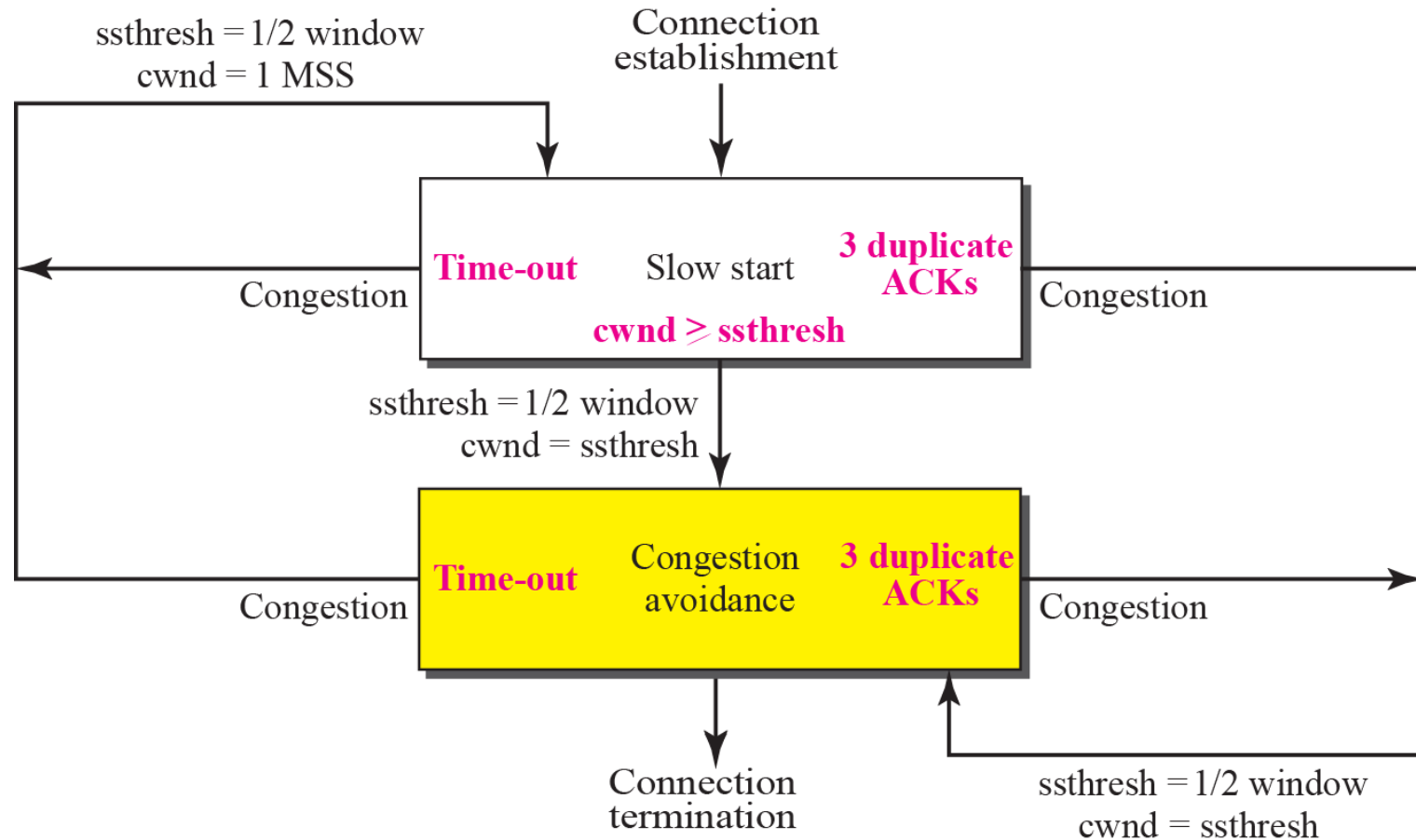


Figure 15.37 *Congestion example*

